UNIVERSITY OF CAMERINO
MASTER DEGREE IN COMPUTER SCIENCE
COMPLEX SYSTEM DESIGN

Evaluation of Bisimulation Metrics on Neural Networks Models for Assessing Semantic Bisimilarity

Members of the Group

Bianca Maria Cerino,

Ibrahim Yaman,

Marco Caputo,

Marco Cocilova

Tutor

Prof. Flavio Corradini

External Tutor

Prof. Marco Piangerelli

ACADEMIC YEAR 2024-2025

Abstract

In formal program verification, bisimulation is a fundamental concept used to assess the equivalence of system models and their specifications. By comparing the state-by-state behavior of complex systems, bisimulation-based methods enable efficient verification and reduction of large-scale models while preserving essential properties. Beyond strict equivalence checking, metrics and pseudo-metrics have been developed to quantify the similarity between systems, offering a structured way to measure their behavioral differences.

This work explores the application of bisimulation theory and model-checking techniques to neural networks, addressing the challenge of defining and computing semantic equivalence and similarity between trained models. We identify suitable bisimulation definitions tailored to the structure and behavior of neural networks, develop a computational library for applying the chosen definitions and metrics on neural networks, and investigate correlations between bisimulation and semantic equivalence or similarity, evaluating the extent to which bisimulation-based metrics reflect the functional closeness of different models.

The proposed framework offers a novel perspective on quantifying neural network similarity beyond traditional comparisons. By adapting bisimulation concepts to neural networks and providing a formal, systematic approach to comparing trained models, we aim to contribute to explainability, model reduction, and robustness analysis in deep learning.

Contents

LIST OF ALGORITHMS vii

LIST OF FIGURES x

I PROLOGUE 1

INTRODUCTION 3

II EXECUTION 5

1 STATE OF THE ART 7
1.1 Background 7
1.2 Related Works 20

2 PROJECT OVERVIEW 25
2.1 Workflow 25
2.2 System Architecture 27
2.3 Technologies 28

3 IMPLEMENTATION 31
3.1 SMT Equivalence Checking 31
3.2 Neural Network Conversion 38
3.3 SPA Behavioral Distance Algorithm 41
3.4 Single Action PFSP Approximate Bisimulation 44

4 RESULTS 47
4.1 Experimental Setup 47
4.2 SMT Approximate Equivalence Metric 49
4.3 Approximate Bisimulation Distance 52

III EPILOGUE 57

CONCLUSION 59

ABBREVIATIONS 61

List of Algorithms

1 Approximate Equivalence Metric algorithm. 36
2 Input-output Approximate Equivalence Checking algorithm. 38
3 Distance Approximation Algorithm, [6] 42

[6] Chen, Han, and Lu, "On metrics for probabilistic systems: Definitions and algorithms"

List of Figures

1.1 Eulero Venn Diagram of Equivalence Hierarchy. 11
1.2 Illustration of the Quotienting Process. 13
1.3 Comparison Between Stutter and Weak Bisimulation. . . 13
1.4 General structure of a multi-layer feed-forward neural network. 16
1.5 ReLU Function. 17
1.6 Sigmoid function. 18

2.1 Gantt Diagram of Project Activities. 25
2.2 Component Diagram of the Developed Library's Static Architecture. 27
2.3 Python's Logo. 29
2.4 Z3's Library Logo. 29
2.5 PyTorch's Logo. 29
2.6 TensorFlow's Logo. 30
2.7 NumPy's Logo. 30
2.8 Pandas' Logo. 30

3.1 Example of a Feed-Forward Neural Network. 32
3.2 Example of argmax modeling of a MPL Neural Network with n hidden states as a Finite State Process. 39
3.3 Example of SMT-based conversion of a Neural Network into a Finite State Process. 40
3.4 Example of conversion of a Neural Network into a Simple Probabilistic Automata obtained through input space sampling. 41

4.1 Results from SMT-Based Metric Applied to Models Trained on Variations of the *Stroke* Dataset. 50
4.2 Execution Times from the SMT-Based Metric Applied to Models Trained on Variations of the *Stroke* Dataset. . . . 51
4.3 Results from SMT-Based Metric Applied to Models Trained on Dissimilar Dataset. 52
4.4 Results from the Maximum Approximate Bisimulation distance applied to models trained on variations of the *Stroke* dataset. 53
4.5 Results from the Average Approximate Bisimulation Distance Applied to Models Trained on Variations of the *Stroke* Dataset. 53

4.6 Execution Times from the Maximum Approximate Bisimulation Distance Applied to Models Trained on Variations of the *Stroke* Dataset. 54
4.7 Execution times from the Average Approximate Bisimulation distance applied to models trained on variations of the *Stroke* dataset. 55

Part I

Prologue

Introduction

In recent years, model checking has undergone significant advancements, driven by the increasing complexity of modern systems and the growing need for automated verification. In general, model checking involves the encoding of a software or hardware system under analysis as a transition system. This model is then checked against a set of properties, typically expressed in temporal logic, to ensure that the system satisfies the desired specifications. One of the most effective methods for comparing models with expected behaviors and reducing system complexity while preserving essential behavioral properties is bisimulation. By collapsing redundant states, this technique not only accelerates verification but also facilitates anomaly detection, where deviations from an expected model can be efficiently identified, making it a valuable tool for assessing system behavior, comparing state trajectories, and ensuring adherence to predefined specifications.

In particular, stutter bisimulation and the approximate variations have gained prominence, as they allow systems to be simplified while maintaining logical equivalence with respect to temporal properties.

In the last few decades, behavioral metrics and pseudo-metrics have been proposed as less strict alternatives to bisimulation, providing flexible ways to quantify the similarity between probabilistic or numerical-labelled models. Moreover, a key trend in this evolution is the shift towards symbolic methods, where the desired system properties and behaviors are encoded into satisfiability modulo theory (SMT) formulas, processed by SMT solvers[1]. To be paired with this model, state-space reduction techniques provided by bisimulations are commonly used in combination with symbolic model checking.

[1] These allow for highly scalable and efficient verification, making it possible to analyze large-scale models that would otherwise be infeasible to check exhaustively.

Inspired by those advancements, few recent projects attempted to adapt model checking techniques to be applied to neural networks (NNs), in order to extend and reproduce those advantages in the field of deep learning models. These techniques are now mainly being used for NN compression, where bisimulation-like reductions help simplify models while preserving key functionalities. Furthermore, input-output equivalences have already been explored to formally compare neural networks, ensuring that different architectures or trained instances produce similar outputs under a variety of conditions. Even if those advancements provide a more rigorous foundation for verifying neural network behavior and improving model robustness, neural networks remain unexplained black boxes, with their decisions often being difficult to interpret with respect to the underlying structure and parameters.

OBJECTIVE

As for complex and distributed systems, looking beyond simple input-output equivalence can provide a more nuanced understanding of neural network behavior, possibly giving insight to the underlying mechanisms that drive model decisions.

Against this backdrop, our work aims to bridge these emerging trends by investigating the application of metrics, bisimulations and model checking techniques to neural networks, with the ultimate aim to improve model explainability. We explore how formal metrics can be used to quantify NN similarity, focusing on both input-output equivalence and behavioral similarity. If such metrics can meaningfully capture the functional closeness of different models, the resulting insights could be used to improve model explainability and generalization, potentially providing a tool for evaluating semantic distance, and so reconstructing a semantic space of neural networks. This, in turn, would enable a deeper understanding of how variations in training data influence model behavior and infer the transferable and fundamental features across different specialized models.

STRUCTURE

Besides the exhaustive exploration of literature in the field, our attempt to tackle this challenge involves an investigation on how the existing bisimulation concepts and metrics can be adapted to neural networks. Indeed, many of the existing bisimulation definitions are tailored to the structure and behavior of traditional labelled transition systems, and may not be directly applicable to NNs, while behavioral metrics and pseudo metrics are often designed for probabilistic systems. For those reasons, we first focus on identifying suitable bisimulation and metric definitions that can be applied to neural networks, providing adaptations between models that are both theoretically sound and practically useful. We then explain the main choices behind the development of our python library, which provides an implementation of the chosen definitions and metrics on NNs. Outside original implementations of algorithms for computing distance metrics, we embed in this library standardized definitions of the models defined in the literature, and implement the conversion of the neural network into such models, which are then used to compute the bisimulation relations. This set of tools has been designed to be flexible and extensible, offering compatibility with the most popular deep learning frameworks and allowing for easy future integrations for different NN architectures. Finally, we used this library to investigate the correlations between the chosen metrics and the concept of semantic equivalence and similarity. We conduct experiments on simple feed-forward neural networks trained on different datasets, characterized by slight semantic deviation of their data, while observing the computational efficiency of the proposed approach. We then present the results of our experiments aiming to evaluate the best bisimulation-based metrics for comparing neural networks and the general feasibility of this novel approach, and discuss the implications of our findings.

Part II

Execution

1

State of the Art

In this chapter, we introduce the fundamental concepts and definitions that will provide the necessary theoretical background for understanding the methodologies and techniques relevant to our research. Afterwards, we will delve deeper into the academic literature, discussing key papers that will highlight the contributions to our development.

1.1 BACKGROUND

In this section we briefly introduce the main elements that are relevant to our report. We provide an overview of bisimulations and neural networks, and we describe the theoretical foundation behind the tools used in this work.

Transition Systems

The definition of transition system (TS) adopted in this document aligns with the one provided by Girard and Pappas [11]:

[11] Girard and Pappas "Approximation Metrics for Discrete and Continuous Systems"

Definition 1 (Transition System)**.** A transition system is defined by the tuple $\mathsf{TS} = \langle Q, \Sigma, \rightarrow, \ll \cdot \gg \rangle$, where:

- Q is a set of states
- Σ is a set of labels σ such that $q \xrightarrow{\sigma} q'$ (σ may be interpreted as an action or as an input)
- $\rightarrow$ is a transition relation $Q \times \Sigma \times Q$
- Q_0 is a set of initial states ($Q_0 \subseteq Q$)
- Π[1] is a set of observations

[1] This usually consists in a set of propositions.

- $\ll \cdot \gg$ is a function $Q \to \{\Pi\}$ used to map an observation π out of a given state q

Eg. $\{q_0 \xrightarrow{\sigma_0} q_1; \ldots; q_1 \xrightarrow{\sigma_n} q_2; \ldots\})$
Eg. $\{\pi_0 \xrightarrow{\sigma_0} \pi_1; \ldots; \pi_1 \xrightarrow{\sigma_n} \pi_2; \ldots\}$

A state trajectory of a TS is a set of elements: $Q \times \Sigma \times Q$

An external trajectory of a TS is a set of elements $\Pi \times \Sigma \times \Pi$ and is accepted by a TS if and only if $\forall i \in \mathbb{N}, \pi_i =<< q_i >>$. The language L(TS) is defined by the set of external trajectories accepted by the TS.

$$\forall external_trajectory \in TS, \exists! state_trajectory \in TS$$

The $Post^{\sigma}(q)$ function defines a set of successors of q.

$$Post^{\sigma}(q) = \{q' \in Q | q \xrightarrow{\sigma} q'\}$$

The $Supp(Post^{\sigma})$ function defines a set of states that don't cause the $Post^{\sigma}$ function to be empty.

$$Supp(Post^{\sigma}) = \{q \in Q | Post^{\sigma}(q) \neq \emptyset\}$$

The Reach(TS) function defines a subset of Π containing only observations π belonging to external trajectories accepted by the TS. This means that the set contains only the reachable observations.

$$Reach(TS) = \{\pi \in \Pi | \exists \{\pi_i \xrightarrow{\sigma_i} \pi_{i+1}\}_{i \in \mathbb{N}} \in L(TS), \exists j \in \mathbb{N}, \pi_i = \pi\}$$

Given two non-deterministic TS A and B, if $(\Sigma_A = \Sigma_B) \wedge (\Pi_a = \Pi_B)$, then the following properties are valid:

$$\begin{array}{ccccc} TS_A \cong TS_B & \to & L(TS_A) = L(TS_B) & \to & Reach(TS_A) = Reach(TS_B) \\ \downarrow & & \downarrow & & \downarrow \\ TS_A \preccurlyeq TS_B & \to & L(TS_A) \subseteq L(TS_B) & \to & Reach(TS_A) \subseteq Reach(TS_B) \end{array}$$

If the two TS are deterministic, the properties are symmetric, substituting every logical implication $(\to)$ with a double logical implication $(\leftrightarrow)$.

Hausdorff Distance for Metric Transition Systems A TS is defined as a metric transition system (MTS) when it is possible to evaluate a distance between two states $q_A \in A$ and $q_B \in B$ and the distance between two observations $\pi_A \in A$ and $\pi_B \in B$, where A and B are two MTS.
The function d used to calculate the distance between two elements is defined as follows:

$$d : A \times B \to \mathbb{R}$$

In order to evaluate the distance between two MTS, we have chosen to implement the Hausdorff distance algorithm as suggested by Girard and Pappas [11].
Also, source [16] showed two versions of the Hausdorff algorithm; one to be used in not-oriented (or symmetric) systems ($h(A, B)$) and the other in oriented (or asymmetric) systems ($H(A, B)$). The not-oriented version of the algorithm consists in iterating every element of A to find the distance with the closest element of B, therefore the greatest between all of those distances in returned as output.

[11] Girard and Pappas "Approximation Metrics for Discrete and Continuous Systems"
[16], *MS Windows NT Kernel Description*

$h : \{Q_q\} \times \{Q_2\} \to \mathbb{R} \qquad h(Q_1, Q_2) = max_{q_1 \in Q_1}\{min_{q_2 \in Q_2}\{d(q_1, q_2)\}\}$

The oriented version, instead, consists in calculating the not-oriented version in both directions and return the greatest result as output.

$H : \{Q_1\} \times \{Q_2\} \to \mathbb{R} \qquad H(Q_1, Q_2) = max\{h(Q_1, Q_2), h(Q_2, Q_1)\}$

Finite State Processes

The finite state process (FSP), defined in detail by Kanellakis and Smolka [13], is a model used to implement concretely a TS. A FSP resembles a nondeterministic finite state automaton (NFSA), having as only difference from a NFSA the presence of labels on the transitions between nodes.

[13] Kanellakis and Smolka "CCS expressions, finite state processes, and three problems of equivalence"

Definition 2 (Finite State Process)**.** For a set of actions Σ, and an unobservable action $\tau \notin \Sigma$, and a set of variables V, we define a Finite State Process (FSP) as a quadruple $\langle K, p_0, \Delta, E \rangle$, where:

- K is a finite set of states
- $p_0 \in K$ is the start state
- $\Delta : K \times V \to 2^K$ is the transition function when $V = (\Sigma \cup \{\tau\})$
- $E : K \to 2^V$ is the extension function

Since a FSP is represented as an automaton[2], every state must be defined by a univocal element $p \in K$.
The transition function Δ returns as output a binary sequence indicating all the possible states that, with the given action $v \in V$, are reachable from the given state $p \in K$.
The extension function E returns as output a binary sequence indicating all the actions that the given state $p \in K$ can perform. Indeed, if the FSP would have $|V| = 1$, permitting no distinctions between transitions, then the definition of the extension function would be equivalent to $E' : K \to V$.

[2] If this is also deterministic, then the definition of the transition function would be equivalent to $\Delta' : K \times V \to K$

Definition 3 (k-observation Equivalence)**.** Two FSP states p, q in the general model are k-observation equivalent, denoted as $p \approx_k q$, if and only if:

- $E(p) = E(q)$ when $k = 0$
- $\forall s \in \Sigma^*$ when $k > 0$:
  - $p \xrightarrow{s} p' \implies \exists q' : q \xrightarrow{s} q' \wedge p' \xrightarrow{s} q'$
  - $q \xrightarrow{s} q'', \implies \exists p'' : p \xrightarrow{s} p'' \wedge q'' \xrightarrow{s} P''$

States p, q are observation equivalent, denoted as $p \approx_k q$, if and only if $\forall k \geqslant 0 \implies p \approx_k q$

Probabilistic Finite States Process

A probabilistic finite state process (PFSP) was developed by our team as part of our research on probabilistic extensions of finite state models. The only difference between an FSP and a PFSP is the definition of the transition function, which in the PFSP is defined as $\Delta : K \times V \to \{(K, P)\}$. Which means that for every action $v \in V$, any state $p \in K$ can have from 0 to n transition towards another state in K. For every state of the PFSP, the sum of the probabilities obtained by the transition function must be equal to 1 (or 0 if the resulting set is empty).

Bisimulation

Bisimulation is a fundamental concept in the study of system equivalence, particularly in the context of transition systems and formal verification. It defines a structural relation between state-based models, ensuring that two systems can simulate each other's behavior while preserving certain properties.

This section explores different equivalence relations based on bisimulation, emphasizing their relevance in probabilistic systems, finite state processes (FSPs), and neural network verification. By analyzing these concepts, we aim to establish a formal foundation for evaluating and optimizing transition systems while allowing for practical trade-offs in computational efficiency and accuracy.

[13] Kanellakis and Smolka "CCS expressions, finite state processes, and three problems of equivalence"

Equivalences Hierarchy Observational and Failure equivalences, introduced by Kanellakis and Smolka [13], are widely used in the context of neural networks. However, for our implementation, an initial research phase was necessary to explore a broader range of options and define a hierarchy between various kinds of equivalence. All following equivalences define the set of FSP couples such that they may be considered equivalent. The function defining the sets is defined as:

$$EQ_e(K^*) = \{(x, y) | x \approx_e y\}$$

where $\approx_e$ is the considered equivalence and K^* is the set of all existing FSP.
Following this, we provide brief definitions of the equivalences selected as candidates for our implementation. Each of these equivalences varies in terms of coarseness and is related to one another through subset relationships, as depicted in the Euler-Venn diagram in Figure 1.1. The equivalences has been divided in two groups of equivalences:

- **Behavioral**: focus on the input/output behavior of the systems.
  - *Contextual Equivalence*: The two automatons may act differently, but their differences must not influence the greater program where they are in case of substitution.

- *Observational Equivalence*: The two automatons can simulate each other and both of them must return the same result when the same input is given.
- *Failure Equivalence*: The two automatons can simulate each other and fail with the same inputs and both of them must return the same result in case of success or the same error in case of failure when the same input is given.
- *Trace Equivalence*: The two automatons must be able to accept the same language. In other words, the initial states of the two automatons must be able to create the same states paths.

- **Structural (Bisimulations)**: establish a difference by analyzing the structure of the TS.
  - *Stuttering Bisimulation*: The two automatons can simulate each other's main functions. Both of the automatons must be able to execute the same operations of the other, but there's tolerance for intermediate operations, additional side-effect operations and loops on visible states.
  - *Weak Bisimulation*: The two automatons can simulate each other. Both of the automatons must be able to execute the same operations of the other, but there's tolerance for intermediate negligible[3] operations and states.
  - *Strong Bisimulation*: The two automatons can simulate each other. Both of the automatons must be able to execute exactly the same operations as the other.
  - *Isomorphism*: The two automatons have exactly the same structure. Nomenclature and notation can be the only difference.

[3] Eg. t-moves.

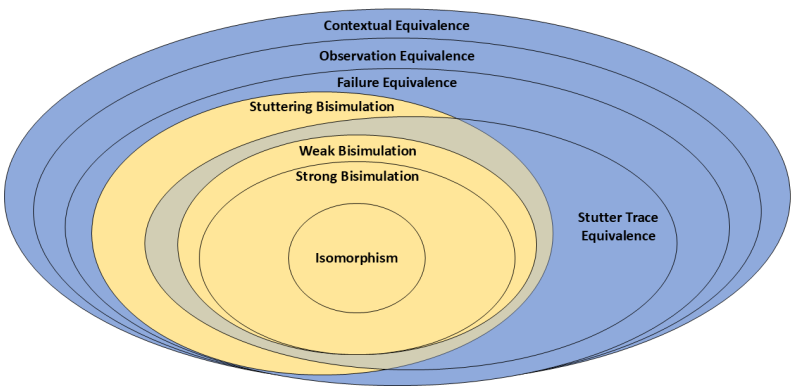

Figure 1.1: Eulero Venn Diagram of Equivalence Hierarchy.

Stuttering Bisimulation A valid candidate for defining equivalence between transition systems is the stuttering bisimulation, meticulously explained by Ranzato and Tapparo [19], since it defines an equivalence based on the structures of two FSP instead of just

[19] Ranzato and Tapparo "Computing Stuttering Simulations"

[19] Ranzato and Tapparo "Computing Stuttering Simulations"

looking for the input-output relations and permits a certain level of variance. The definition given by Ranzato and Tapparo [19] of the stuttering bisimulation is as follows:

Definition 4. Let $K = (\Sigma, \rightarrow, L)$ be a Kripke Structure (KS) such that Σ is the set of all the process' states, $\rightarrow \subseteq \Sigma \times \Sigma$ is the transition operation and $L : \Sigma \rightarrow \wp(AP)$, with $\wp(AP)$ representing a set of atomic propositions. A relation $R \subseteq \Sigma \times \Sigma$ is a divergence blind stuttering bisimulation on K if for any $s, t \in \Sigma$ such that $(s, t) \in R$:

- $L(s) = L(t)$
- If $s \rightarrow s'$ then there exist $T_0, \ldots T_K \in \Sigma$, with $k \geqslant 0$, such that:
  - $t_0 = t$;
  - $\forall i \in [0, k), t_i \rightarrow t_{i+1}) \wedge (s, t_i) \in R$;
  - $(s, t_k) \in R$.
- If $t \rightarrow t'$ then there exist $S_0, \ldots S_K \in \Sigma$, with $k \geqslant 0$, such that:
  - $s_0 = s$
  - $\forall i \in [0, k), s_i \rightarrow s_{i+1}) \wedge (s_i, t) \in R$
  - $(s_k, t) \in R$

A weak bisimulation, instead, would tolerate the presence of the b_1 state only if all of its ingoing and outgoing transitions are considered τ-transitions.

A transition $s \rightarrow s'$ is called *"stutter step"* when $L(s) = L(s')$, which means that s and s' are two states with the same attributes. So, when a stutter step occurs, the system shifts from a state to another equivalent one. Obviously, if a transition $s \rightarrow s'$ is a stutter step, the two states can be fused into one unique state, turning the automaton into a simpler yet *"stutter equivalent"* one.

[4] Baier and Katoen *Principles of model checking*

Stuttering Quotient System Building on the previously discussed concepts as a knowledge base and analyzing Baier and Katoen [4] on stutter bisimulation, we have derived the following definition for FSP compression using the stutter quotient system:

Definition 5 (Finite State Process Compression)**.** A finite state process compression can be defined as:

$$TS/\approx = (S/\approx_{TS}, \{\tau\}, \rightarrow_{\approx}, I_{\approx}, AP, L_{\approx})$$

where:

- $I_{\approx} = \{[s]_{\approx} | s \in I\}$
- $\rightarrow_{\approx}$ is defined by $\frac{s \xrightarrow{\alpha} s' \wedge s \not\approx s'}{[s]_{\approx} \xrightarrow{\tau}_{\approx} [s']_{\approx}}$
- $L_{\approx}([s]_{\approx}) = L(s)$

This notation expresses the process of grouping the states of the system using the equivalence criteria. In this way it is possible to simplify the automaton by reducing its states into groups of equivalence, creating a new stuttering equivalent automaton as illustrated in Figure 1.2.

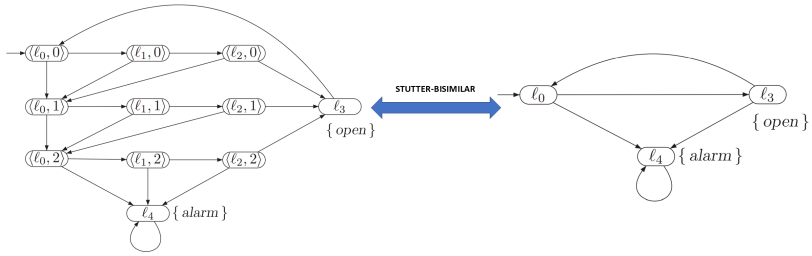

Figure 1.2: Illustration of the Quotienting Process.

Proposing an example, illustrated by Figure 1.3, given two FSP A and B such that $L(a_0) = L(b_0) \wedge L(a_f) = L(b_f)$, with $a_i, a_f \in A$ and $b_i, b_f \in B$, if the initial state a_0 can transit directly into a_f, the state b_0 can simulate a_0 even if it requires going to a state $b_1 \in B | L(b_0) = L(b_1)$ and then back to b_0.

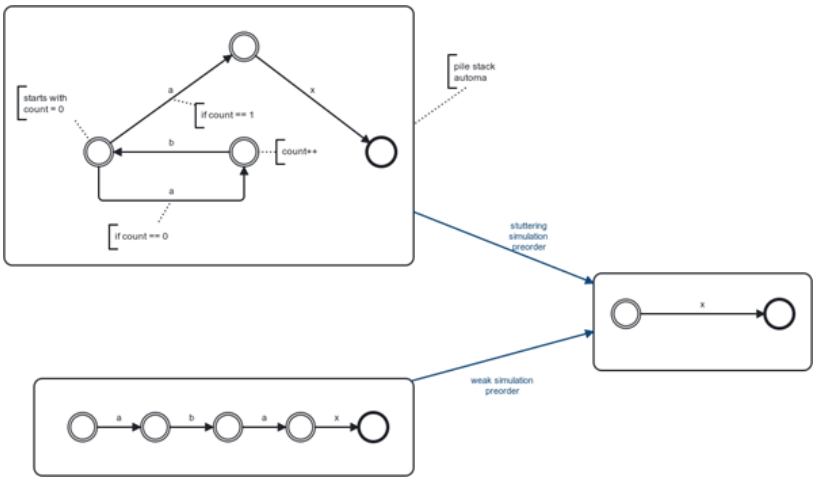

Figure 1.3: Comparison Between Stutter and Weak Bisimulation.

Probabilistic Approximate Bisimulation

Another valid candidate, not mentioned in the hierarchy graph, is the approximate bisimulation, which has been introduced by Girard and Pappas [11], as a relation between two transition systems that ensures their behaviors remain similar within a bounded error margin, considering both the differences in outputs and the state transitions under the same inputs and context. This kind of bisimulation can't be applied, as it is described by the article, for feed-forward neural network comparisons due to two main reasons: lack of attributes and structure regularity.

[11] Girard and Pappas "Approximation Metrics for Discrete and Continuous Systems"

Lack of Attributes The main issue with this solution was that neural network nodes, apart from their layer and arbitrary names, lack distinguishing attributes. This situation makes it impossible to establish a proper state correspondence based on state's labels equivalence, which is essential for the approximate bisimulation algorithm to evaluate whether two states from different transition systems are transitioning to two sets of equivalent states or not.

So, in order to adapt the model used, the PFSP has been deduced using the combination of target state and probability to reach it as an identifier, allowing the transition system to provide the observation set Π based on transitions' probabilities.

Structure Regularity The regularity of feed-forward neural networks' structures is more of an optimization opportunity than an issue. Indeed, having every state in any layer transit only to a state in the successive layer is resource-demanding. To fix this, we developed an optimization[4] for said algorithm by making it perform recursive deep searches and linear iterations through layers, using a search matrix as memorization.

[4] More details about this can be found in Subsection 3.4 - "*Search Matrix and Optimizations*"

This approach is very efficient for detecting approximate probabilistic bisimulations with a computational weight between $O(2n)$ and $O(3n)$, having little difference from best to worst cases. The high efficiency is given by the use of a search matrix: it is used as a memory capacity in the recursion[5] and it is filled by the first operation[6]. This allows the execution of the Hausdorff algorithm to be distributed across different layers, instead of being performed on all states simultaneously.

[5] Used to avoid repeating the same operations twice and to decrease the memory required by the recursion stack.
[6] Makes iterations become highly efficient from a computational perspective.

Using layers as "clusters" is particularly advantageous for two main reasons:

1. The Hausdorff algorithm is not linear and exhibits a complexity of $O(2nm)$, making it computationally expensive when applied to a large state space.

2. The candidate nodes for equivalence are strongly bisimilar[7], reducing redundant calculations.

[7] The comparison is effectively restricted to pairs of layers between two neural networks

Neural Networks

Neural networks are among the most elegant programming paradigms ever devised. In the traditional approach to programming, we instruct the computer on precisely what to do, breaking down complex problems into numerous small, well-defined tasks that the machine can easily execute. In contrast, a neural network does not rely on explicit instructions to solve a problem. Instead, it learns from observational data and autonomously develops a solution tailored to the given problem as described by Nielsen [17]. Artificial neural networks are designed to simulate the human nervous system for machine learning tasks by modeling computational units in a manner similar to biological neurons. The grand vision depicted by Aggarwal [2], regarding artificial neural networks, is to create artificial intelligence by building machines whose architecture mimics the computational processes of the human nervous system. *Deep Neural Networks* (DNNs), outlined by Xiang and Shao [24], have emerged as a foundational technology in a wide range of contemporary fields, including image processing, pattern recognition, adaptive control, and autonomous systems. They have demonstrated exceptional ca-

[17] Nielsen *Neural Networks and Deep Learning*

[2] Aggarwal *Neural Networks and Deep Learning: A Textbook*

[24] Xiang and Shao "Approximate Bisimulation Relations for Neural Networks and Application to Assured Neural Network Compression"

pabilities in addressing complex, non-linear problems that often lie beyond the reach of conventional analytical or heuristic methods. Their capacity to autonomously learn hierarchical representations from high-dimensional data has established them as indispensable tools for solving intricate real-world challenges. To further enhance their effectiveness in navigating complex tasks and diverse operational environments, recent research has increasingly focused on the development of more advanced neural architectures, marked by greater scale and structural complexity. These advancements aim to optimize model performance, improve generalization, and increase precision, thereby expanding the frontiers of computational intelligence in both theoretical and applied domains. The success of deep neural networks is attributed to three factors: access to large datasets, the development of new optimization algorithms and model architectures, and the increased availability of computational resources. Notably, the latter two have enabled researchers to build deeper networks, consistently achieving state-of-the-art results in various complex machine learning tasks. However, as we can see from Wiedemann et al. [23], this has led to an exponential increase in the number of parameters required by these models, making them increasingly memory-intensive and introducing significant challenges beyond just higher memory demands. To address the challenges of complexity and computational resource demands in neural networks, it may be necessary to simplify these models. Approximate bisimulation presents a potential solution for achieving this goal.

[23] Wiedemann et al. "DeepCABAC: A Universal Compression Algorithm for Deep Neural Networks"

For the scope of this work, we adopt the following definition of neural network, which captures the fundamental structural elements of interest of multi-layer feed-forward neural networks, the main object of study of the project.

Definition 6 (Neural Network, Prabhakar [18]). A neural network (NN) is a tuple:

$$N = \left(k, Act, \{\mathcal{S}_i\}_{i\in[0,k]}, \{W_i\}_{i\in(0,k]}, \{b_i\}_{i\in(0,k]}, \{A_i\}_{i\in(0,k]}\right),$$

where:

- k represents the number of layers (excluding the input layer);
- Act is a set of activation functions
- For each $i \in [0, k]$, $\mathcal{S}_i$ is a set of nodes of layer i, where we assume $\mathcal{S}_i \cap \mathcal{S}_j = \emptyset$ for $i \neq j$
- For each $i \in (0, k]$, $W_i : \mathcal{S}_{i-1} \times \mathcal{S}_i \to \mathbb{R}$ is the weight function that captures the weight on the edges between nodes at layer $i-1$ and i
- For each $i \in (0, k]$, $b_i : \mathcal{S}_i \to \mathbb{R}$ is the bias function that associates a bias with nodes of layer i
- For each $i \in (0, k]$, $A_i : \mathcal{S}_i \to Act$ is an activation association function that assigns an activation function to each node of layer i

Figure 1.4 illustrates the general structure of a NN as intended in the definition and as we will intend from now throughout this report. However, whenever we consider neural networks as black-box mappings between inputs and predicted output, we may treat neural networks as functions $N : I \to O$, where $I \subseteq \mathbb{R}^n$ is some input domain with $n = |\mathcal{S}_0|$ the number of features, and $O \subseteq \mathbb{R}^m$ is some output domain with $m = |\mathcal{S}_k|$ the number of output neurons.

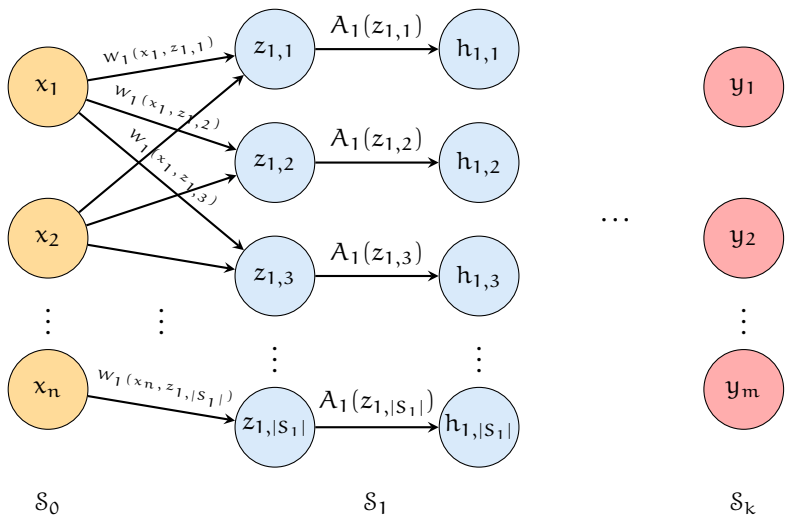

Figure 1.4: General structure of a multi-layer feed-forward neural network.

Activation Functions in Neural Networks Understanding the role of activation functions in artificial neural networks is crucial for comprehending how these networks operate, as well as for evaluating their computational requirements and complexity. In artificial neural networks, different layers consist of interconnected neurons. The outputs/information from the inner layers are passed on to subsequent layers, ultimately leading to the generation of the final result by the outermost layer (the output layer). The output passed to the outer layer is modified by the application of activation functions, which introduce nonlinear characteristics, enabling the nonlinear processing of data produced by the inner layers. Activation functions play a critical role in the learning process of artificial neural networks, as they facilitate the modeling of complex and nonlinear relationships between inputs and outputs. This allows the network to uncover abstract patterns and meaningful relationships within the data, as told by Sharma, Sharma, and Athaiya [20]. Activation functions are specifically used in artificial neural networks to transform an input signal into an output signal. This output signal is then passed on as input to the subsequent layer. This process ensures the flow of information through the layers, playing a critical role in enabling the network to learn complex relationships (nonlinear patterns) within the data. Activation functions overcome the limitations of linear

[20] Sharma, Sharma, and Athaiya "Activations Functions in Neural Networks"

Without activation functions, neural networks would only be capable of modeling simple linear relationships, thereby losing the ability to provide solutions to complex real-world problems. This behaviour is defined as a Linear Regression Model, which typically has limited performance and flexibility.

equations by introducing nonlinear behavior into neural networks, enabling them to learn abstract data relationships across layers. Artificial neural networks require nonlinear activation functions to learn complex relationships within data and create nonlinear functional mappings between inputs and outputs. These functions provide dynamism to the network, enabling it to extract abstract patterns from the data and model intricate relationships. Furthermore, the most critical characteristic of activation functions is their differentiability[8].

[8] This allows for the computation of errors with respect to the weights (via backpropagation) and the updating of weights using optimization techniques like Gradient Descent, thereby minimizing errors effectively.

Linear Activation Function The linear activation function is a function directly proportional to the input and is expressed as $f(x) = \alpha x$, where α is a constant determined by the user. Unlike the binary step function, its derivative is non-zero (constant α), which ensures a non-zero gradient and allows for weight updates during backpropagation. However, the constant gradient leads to the same update factor being applied at each iteration, which hinders error optimization and limits the network's ability to learn complex patterns within the data. As a result, the linear activation function is primarily used in simple tasks, such as modeling linear relationships, or in situations where interpretability is crucial. While it lacks flexibility and is ineffective for complex problems, it can be useful for understanding theoretical concepts or for simple regression models.

ReLU Activation Function The ReLU (Rectified Linear Unit) activation function is a widely used nonlinear function in neural networks, defined mathematically as $f(x) = \max(0, x)$, where it is linear for positive inputs and outputs zero for negative inputs. One of its key advantages is efficiency, as only neurons that produce positive outputs are active, leading to sparse activation and reduced computational costs.

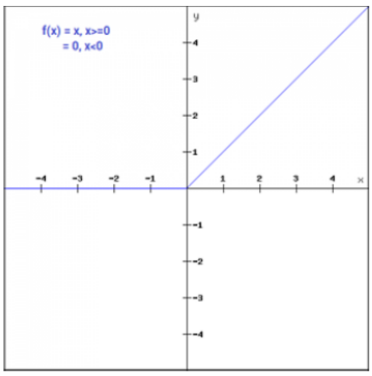

Figure 1.5: ReLU Function.

Additionally, ReLU helps alleviate the gradient problem, as it maintains a constant gradient (a derivative equal to 1) in the positive region, preventing the issue of vanishing gradients during backpropagation. However, it also has some drawbacks, including the *"dead ReLU"* problem, where neurons with negative inputs have zero gradients and, as a result, their weights are not updated, potentially rendering them permanently inactive. Furthermore, its linear behavior around zero may not capture more complex patterns in some cases. Despite these limitations, ReLU can be considered to be the most used piecewise-linear activation function in deep neural networks, particularly in convolutional neural networks (CNNs) and large-scale models, due to its computational efficiency and effectiveness in mitigating gradient-related issues.

Sigmoid Activation Function The Sigmoid Activation Function is one of the most commonly used activation functions due to its non-linear nature. The sigmoid function maps input values to a range between 0 and 1. It is defined as follows:

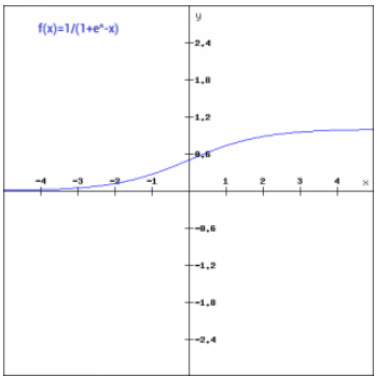

Figure 1.6: Sigmoid function.

$$f(x) = \frac{1}{1 + e^{-x}}$$

This property makes it suitable for binary classification tasks, where the output can be interpreted as a probability, so that binary classificators can use a single output neuron with such an activation function. The sigmoid function is continuously differentiable and exhibits a smooth, S-shaped curve. Its derivative is given by:

$$f'(x) = f(x)(1 - f(x))$$

However, the sigmoid function is not symmetric around zero, which may lead to neuron outputs having the same sign. This issue can be mitigated by rescaling the sigmoid function.

Metrics and Pseudo-metrics

The following definitions are those of the mathematical tools used to measure distances or differences between elements in various domains. Specifically, in neural networks and bisimulation-based systems, said elements are identified as states.

Definition 7 (Metric)**.** A metric function $d : X \times X \to \mathbb{R}$ on a set X satisfies:

- $d(x, y) = d(y, x)$
- $d(x, z) \leqslant d(x, y) + d(y, z)$
- $d(x, y) = 0 \iff x = y$

Definition 8 (Pseudo-metric)**.** A pseudo-metric function $d : X \times X \to \mathbb{R}$ on a set X satisfies:

- $d(x, y) = d(y, x)$
- $d(x, z) \leqslant d(x, y) + d(y, z)$
- $d(x, y) \geqslant 0$

Thus, a pseudo-metric space allows elements to be "indistinguishable" in terms of distance without being strictly identical. Also, they further allow to reason about states based on what it is already known about other similar states. Those are used in formal verification, concurrency theory, and in safe reinforcement learning to provide (non-)reachability guarantees, but can also be useful for state aggregation and abstraction.

Satisfiability Modulo Theory

Satisfiability Modulo Theory (SMT) is the problem originated by the extension of the Boolean Satisfiability Problem (SAT) by determining the satisfiability of first-order formulas under specific logical theories, such as arithmetic, arrays, or equality.

SAT, the first problem proven NP-complete, has seen significant advances in solver efficiency since the mid-1990s, leading to its integration in model checking. However, the trade-off between the expressiveness of a logic and the ability to automatically check the satisfiability of its formulas has been considered a central issue in formal verification. A practical compromise have been achieved with fragments of first-order logic that are restricted either syntactically, for instance by allowing only certain classes of formulas, or semantically, by constraining the interpretation of certain function and predicate symbols, or both. When semantic restrictions are involved, they can be understood as limiting the interpretations of certain symbols to models of some logical background theory and, in such cases, we speak of Satisfiability Modulo Theories (SMT).

Definition of SMT From another perspective, SMT is born from the introduction of Many-sorted logic (MSL)[9], an extension of first-order logic that allows multiple "sorts" (or types) of variables. Further extensions like Order-sorted logic (OSL)[10] and First Order Logic with Parametrical Types (FOLPT)[11] have been introduced to enhance flexibility and expressiveness, making formulas closer to the way they are expressed in programming languages.

Formally, given a Σ-signature $\Sigma = \langle \Sigma^S, \Sigma^P, \Sigma^F, m_P, m_F \rangle$, respectively defining the set of sorts, predicate and function symbols, along with the mapping of symbols to their respective arities and sequence of sorts, a Σ-interpretation $\mathcal{A}$ of Σ over a set of variables X, whose sorts are in Σ^S provides a semantic interpretation of symbols in Σ to elements of sort domains. In this sense, $\mathcal{A}$ can satisfy a formula φ, denoted as $\mathcal{A} \models \varphi$, if the formula evaluates to true under the interpretation $\mathcal{A}$. A Σ-model is a Σ-interpretation where X is empty, so is a way of assigning meaning to the symbols in a signature Σ (sorts, functions, and predicates) without any specific free variables. Adding variables enables the Σ-model to evaluate and check the truth of specific expressions, making it possible to determine satisfiability (whether a formula is true in that model).

[9] Instead of a single domain of discourse, elements are divided into different, distinct domains, each associated with a specific sort, without requiring explicit type constraints in formulas. For instance, supposing to have a sort for people and one for books, predicates specific to each sort can be defined, like isAuthorOf(x, y) meaning "person x is the author of book y".

[10] Allows to define a hierarchy of sorts, where a sort can be a subtype of another sort like in object-oriented programming.

[11] Allows variables, functions, and predicates to be parameterized with types as with parametric polymorphism.

Definition 9 (Σ-Theory, [7]). A Σ-theory T is a pair (Σ, A) where:

- Σ is a Σ-signature.
- A is a class of Σ-models.

So a Σ-theory is essentially a set of Σ-models that all interpret symbols in the same signature Σ according to some shared constraints or rules. Theories can be seen as constraints on interpreting models, as A expresses the set of models that satisfy the specific constraints of the theory.

Definition 10 (Satisfiability Modulo Theory, [7]). Given a Σ-theory $T = (\Sigma, A)$ and a formula φ in the language of Σ, the Satisfiability Modulo Theory problem is to determine whether there exists a Σ-model $\mathcal{A} \in A$ such that $\mathcal{A} \models \varphi$.

So we call a SMT solver any program that tries to determine the satisfiability of some class C of formulas in some theory T. Whenever a formula is satisfiable, the solver returns a model that satisfies the formula, or a proof that the formula is unsatisfiable. In the end, instead of purely propositional reasoning, SMT constrains interpretations of functions and predicates according to predefined theories, enabling more precise modeling of real-world systems while maintaining automated resolvability.

SMT Solvers in Model Checking SMT solvers have been used in a variety of applications, including software verification, static analysis, type checking and optimization. However one of the most relevant and widespread applications of SMT solvers is in bounded and unbounded model checking, where they are used to verify properties of finite and infinite state transition systems.

In bounded model checking (BMC), a transition system is unwound up to a bound k, constructing a formula that is satisfiable if a counterexample exists within that depth. SAT solvers efficiently handle these formulas, encoding path constraints in transition systems. Formally, let S be a transition system, let $I[x]$ be the formula satisfied exactly by the initial states of S, let $T[x, y]$ be the formula satisfied by all pairs of states (x, y) such that y is a successor of x in S, and let $P[x]$ be the state property to be checked. Bounded Model Checking tries to prove that that the formula

$$I[x_0] \wedge \bigwedge_{i=0}^{k-1} T[x_i, x_{i+1}] \wedge \bigvee_{i=0}^{k} \neg P[x_i]$$

is T-satisfiable according to some theory T. However, BMC is inherently incomplete, as counterexamples requiring more than k steps remain undetected.

SMT solvers play a fundamental role in unbounded model checking, allowing verification techniques like inductive proofs, interpolation, and abstraction refinement to prove correctness without a fixed bound.

Unbounded Model Checking, on the other hand, aims to prove that a property holds for all reachable states of a system, without unwinding the transition system up to a fixed bound.

1.2 RELATED WORKS

In this section we provide an overview of the related published works in the field of bisimulation metrics and neural networks, describing the recent relevant approaches combining machine learning-based techniques and bisimilarity. We review a set of articles that provide definitions of metrics based on bisimilarity, and other recent works that describe related innovative approaches in the field of deep learning. All selected articles provided useful insights on the scope of the research and helped to identify the main challenges and opportunities in the field.

Approximate Bisimulation for Neural Networks

[18] Prabhakar "Bisimulations for Neural Network Reduction"

The work written by Prabhakar [18], explores the concept of re-

ducing the size of neural networks while preserving their functional equivalence through the mathematical framework of bisimulation . Neural networks often have large and complex structures that make analysis and verification challenging. Bisimulations provide a formal way to reduce these networks[12] without altering their input-output behavior. NN-bisimulation is introduced as a way to define equivalence between neural networks by grouping nodes with similar properties (biases, activation functions, and sums of incoming weights). This allows creating a smaller, semantically equivalent network. This reduction process is inspired by classical bisimulation concepts in system verification but adapted for neural networks. Since strict equivalence can limit the amount of reduction, the authors extend the concept to approximate bisimulation, which allows a small bounded deviation (ϵ) between the reduced and original network. This offers a balance between the size of the reduction and the precision of the network's behavior Prabhakar [18] propose algorithms for constructing the smallest bisimulation-equivalent network and checking for approximate bisimulations. The complexity of these algorithms depends on the number of nodes and edges in the network but is feasible for practical use.

[12] The reduced network is smaller and easier to analyze and explain, which is especially useful in safety-critical applications like autonomous systems.

Behavioral metrics and pseudo-metrics

One of the first contributions in literature on the definition of less rigid metrics of bisimilarity is provided by Deng et al. [8], where the definition of a flexible metric of bisimulation for quantitative systems is proposed. Quantitative systems are systems such as probabilistic or resource-dependent systems, where traditional bisimulation fails to account for small variations in quantities associated with transitions, like probabilities or costs. In this work, metrics[13] to measure the difference between processes that are not exactly bi-similar are defined and explored. A rigid approach may, in fact, result in treating processes with minor differences as entirely distinct, which is impractical for real-world applications like security systems where slight deviations are tolerable. The authors extend previous work on probabilistic systems and Markov chains by using the Hutchinson metric to define state-metrics for a generalized framework[14] called Action-labelled Quantitative Transition Systems (AQTS).

[8] Deng et al. "Metrics for Action-labelled Quantitative Transition Systems"

[13] Borrowing the concept of metric from mathematics, indeed provides a flexible way to quantify the "distance" between processes.

[14] This encompasses various quantitative systems such as probabilistic automata, weighted automata, and reactive models, this generalization is rather useful because it can be used and expanded also to other systems, such as neural networks.

Another significant contribution to the study of flexible bisimulation metrics comes from Chen, Han, and Lu [6], this paper extends the study of behavioral distances in probabilistic systems previously mentioned, refining the notion of bisimulation by introducing pseudometrics that capture gradual behavioral differences instead of rigid equivalences. They propose a framework for defining behavioral pseudometrics for simple probabilistic automata (SPA), emphasizing the need for quantitative measures in systems where nondeterminism and probability coexist. Unlike classical bisimulation, which treats any difference in transition probabilities as a complete mismatch, these allow for a smooth, numerical differentiation between system

[6] Chen, Han, and Lu "On metrics for probabilistic systems: Definitions and algorithms"

states.

This is done to have a *range* of similarity instead of a classification as equivalent/non-equivalent.

One of the central contributions of the paper is the introduction of a **fixpoint characterization**, where the metric is defined as the greatest fixpoint of a monotonic functional on the lattice of state metrics. Additionally, the authors address the computational challenges of the evaluation, proposing an approximation algorithm to compute the distance between two states within a desired level of accuracy. This leverages Tarski's decision procedure and a binary search strategy to iteratively refine computations in the transition systems, providing a practical approach to a previously open problem.

[5] Castro "Scalable methods for computing state similarity in deterministic markov decision processes"

Notions of behavioral similarity are explored by Castro [5] via state pseudometrics, considering deterministic Markov Decision Processes (MDPs) and those pseudometrics defined on couples of states to real values. Authors also propose an architecture which employs neural networks to capture the similarity between states even in large or continuous state spaces, defining a differentiable loss function for learning an approximation of the two bisimulation metrics using neural networks. The approach aims to approximate the similarity (or distance) between states in terms of their behavior in the MDP, and the neural network model uses sampled transitions to iteratively update its estimate of the bisimulation distance.

Experimental examples are proposed, evaluating bisimilarity between states of agents in computer games. For example, two states that lead to similar outcomes in the game (e.g., similar rewards or transitions) demonstrate to have a low bisimulation distance.

Applications of SMT solvers

Besides their extensive use in formal verification and model checking, SMT solvers have been employed in a variety of applications, including those related to neural networks and deep learning.

[3] Albarghouthi "Introduction to Neural Network Verification"

As explained by Albarghouthi [3], one relevant application in this field is the SMT-based verification of neural networks, which attempts to provide formal guarantees about the behavior of neural networks by encoding desired properties into logical constraints. The work by Katz et al. [14] represents an example of this approach, where the authors propose an SMT-based verification tool for deep neural networks. Properties of neural networks are encoded as logical formulas involving input-output relations, and the tool uses an SMT solver to determine whether the constraints are satisfiable. Is important to note that input-output relations are relevant not only for verification of correctness, but also for the comparison of neural networks, which is the main focus of our work.

[14] Katz et al. "Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks"

[10] Eleftheriadis et al. "On Neural Network Equivalence Checking Using SMT Solvers"

One recent work in this area particularly relevant to our research, is the one by Eleftheriadis et al. [10], where an SMT-based approach to verifying the equivalence of neural networks is presented. The authors define different equivalence criteria, including strict functional equivalence and approximate measures based on L_p norms and output order preservation. They propose encoding the equivalence checking problem into logical satisfiability constraints, allowing SMT solvers to determine whether two networks produce comparable outputs. This work is one of the first attempts at a general-purpose SMT formulation for neural network equivalence, distinguishing it-

self from prior research that primarily focused on heuristic similarity measures. As in our work, this methodology serves as a foundation for further research into formal methods for neural network semantical comparison.

However, limitations of the approach are also considered, particularly the scalability challenges posed by large neural networks. Through experiments on classification and regression models, the authors assess the feasibility of SMT-based equivalence checking and highlight scenarios where the approach becomes computationally intractable. Their findings suggest that while SMT solvers can provide formal guarantees, their applicability remains limited to small-scale models due to the exponential growth in constraints. Another important aspect is the limited applicability of the approach to networks with non-linear activation functions, which are not directly supported by linear SMT solvers.

This last observation has been further investigated by other works, either by relying on the use of more advanced SMT solvers that can handle non-linear constraints [12], or rather by proposing linear approximations techniques of activation functions [15]. In such cases, the SMT-guided verification approach can be extended to a more general class of NNs, known as recurrent neural networks, where nonlinear activation functions such as Sigmoid and Tanh cannot be avoided.

[12] Guidotti, Pandolfo, and Pulina, "Verifying Neural Networks with Non-Linear SMT Solvers: a Short Status Report"

[15] Mohammadinejad et al., "DiffRNN: Differential Verification of Recurrent Neural Networks"

Another important contribution is brought by Abate, Giacobbe, and Schnitzer [1], where a novel data-driven approach to computing finite bisimulations for state transition systems with large or infinite state spaces is presented. Overall, the paper provides a methodology for using machine learning and SMT solvers for bisimulation computation, which provides advantages in the case of the verification of very large or infinite-states complex systems. The approach relies on SMT solver to ensure that learned abstractions generalize across the entire state space. The proposed method employs a counterexample-guided inductive synthesis (CEGIS) loop to refine the learned bisimulations. A Binary Decision Tree (BDT), with parametric linear predicates at each decision node, is initially set with a group of sample states, combined with the related Linear Temporal Logic expression. The learner proposes the function computed by the model to the verifier, a SMT solver. When the SMT solver detects a violation (a counterexample in the original labeled transition system), the classifier is updated, along with the ranking functions associated to each partition (classification) of states, ensuring that the abstraction is progressively improved until a valid bisimulation is achieved.

[1] Abate, Giacobbe, and Schnitzer *Bisimulation Learning*

This work represents a successful attempt in laveraging the counterexamples produced by an SMT solver, which in general could be used to refine a machine learning classifier in a CEGIS loop. In the case of neural networks and SMT-based equivalence checking considered in our work, this approach could be used to train a neural neural network considered unequivalent to another one of reference.

Indeed, by iteratively providing counterexample inputs to the network, the model could be trained to become closer to the reference model, thus providing a range of parameter configurations that could be used to infer a semantic space of neural networks.

Probabilistic Equivalence

[9] Desharnais, Laviolette, and Zhioua "Testing probabilistic equivalence through Reinforcement Learning"

In the work made by Desharnais, Laviolette, and Zhioua [9] we can find an instance of a reinforcement learning (RL)-based approach to test probabilistic equivalence between processes when no model is available. By defining a Markov Decision Process (MDP), this method quantifies the distance between two processes through a divergence metric. A new family of equivalences, called "K-moment"[15], is introduced, which is stricter than trace equivalence but less stringent than bisimulation. The approach allows comparing processes with an unknown model through repeated tests and computations using Q-Learning algorithms, without requiring recursive state replication, thus enabling the analysis of large-scale systems.

[15] Probabilistic equivalence that assesses similarity between labelled Markov processes (LMPs), restricted to countable ones, by comparing their probabilistic behavior across multiple "moments." Each moment represents a level of equivalence; for instance, 1-moment corresponds to trace equivalence, while higher moments offer more stringent comparisons.

By modeling the problem as an MDP, the authors use reinforcement learning to estimate divergence between processes without needing an explicit model of the implementation. They define a game-like scenario where an RL agent interacts with both a "Specification" (the expected process behavior) and an "Implementation" (the actual process) to detect discrepancies. Rewards are structured to measure divergence, assigning high rewards for observed behavioral differences between Specification and Implementation, while normalizing for probabilistic fluctuations. This approach ensures that the determination of equivalence is based on an aggregated evaluation rather than isolated discrepancies.

There's also a confrontation with a "Clone", this is an exact replica of the "Specification" and it helps discern when a discrepancy can be attributed to probabilistic fluctuations rather than actual differences in behavior.

2

Project Overview

In this chapter we provide a detailed description of the development process, illustrating the general idea behind the project, the high-level description of the workflow, the static architectural design of the system as well as the main technologies used to develop the various components of the system.

2.1 WORKFLOW

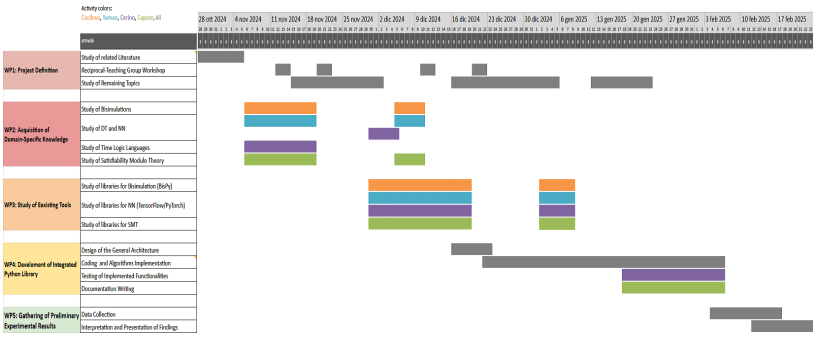

Figure 2.1: Gantt Diagram of Project Activities.

The project has been mainly developed following a waterfall model, with the inclusion of some practices from the iterative methodologies. As evidenced in the Gantt diagram in Figure 2.1, the project has been divided into several macro-phases called *work packages* (WP), each one of those defined by its own set of goals. We further divided each work package in activities that can be iterated multiple times to achieve the desired results on the produced deliverables, the different colors of the activity bars represent the member of the group responsible for the activity, with the gray tonality representing the activities carried out by the whole group.

We provide a description of the main work packages below:

For instance, the reciprocal-teaching group workshops, consisted in explanations and discussions of specific topics that were object of study in the project, in order to facilitate and accelerate the process of learning among all the members of the group.

- **WP1:** *Project Definition and Planning* - This work package focused on the definition of the project goals and the planning of the activities to be carried out. The main objective consisted in the study of the state of the art, the definition of the project requirements with the tutor and external tutor, and the planning of the other project activities, including topics to be studied and resources to be used. We include in this section all those activities not directly related to the development of the system that have been iterated over time through following work packages.

- **WP2:** *Acquisition of Domain-Specific Knowledge* - The primary goal of this work package was the acquisition of the domain-specific knowledge required to develop the project and proposed method. The main activities consisted in the study of specific topics, which included bisimulation relations, machine learning models and neural networks, model checking, time logic languages and satisfiability modulo theories. During this phase each member of the group specialized in a specific topic, in order to facilitate the overall development. The first deliverables of the project were produced during this phase, consisting in state of the art reports and specific papers reviews.

This also helped in identifying the currently publicly available tools and technologies in the field of model checking, evidencing the need to develop an open-source library.

- **WP3:** *Study of Existing Tools* - The main focus of this work package was to discover and study the existing tools that could be used to develop the proposed method. The activities of this phase consisted in the study of the chosen programming language, the existing libraries for SMT solving and those for machine learning, with the particular concern for neural network models. The latter, in particular, revealed its utility in the subsequent phase: an in-depth study of the neural network representation among different frameworks was necessary for making the final result compatible with the most popular libraries. We have also considered this step to be preparatory for the initial static design of the system.

- **WP4:** *Development of Integrated Python Library* - The main objective of this work package was the design and development of the Python library, which we consider to be the core of the project. In the first phase of design we have identified the main components of the library and their interaction, as illustrated in Subsection 2.2. Once the loosely coupled components were identified, we proceeded with the actual development of the library, following the design choices made. The library was developed in an iterative way, with the implementation of the components being carried out in parallel among members of the group. Right after the first prototypes, the code underwent many instances of refactoring, due to inherent difficulties in the implementation of the algorithms. Activities of testing and documentation writing were carried out along with the development of the library.

- **WP5:** *Gathering of Preliminary Experimental Results* - The last work package consisted in the definition, execution and analysis of the preliminary experiments, aimed at evaluating the proposed metrics and algorithms. The main activities consisted in the search and processing of the datasets, the training of the neural network models, the setup and execution of the experimental trials, the plotting of the results and the analysis of the obtained data. These tasks have been carried out as collaborative activities, with the members of the group working simultaneously on different parts of the process.

2.2 SYSTEM ARCHITECTURE

In this section, we'll provide an high-level overview of the architecture of the developed library as illustrated in Figure 2.2. As can be seen from the overall structure, the library's main functionalities all revolve around the information regarding some given neural networks in input, the components in cyan have been developed in this project, while the ones in violet have been externally imported.

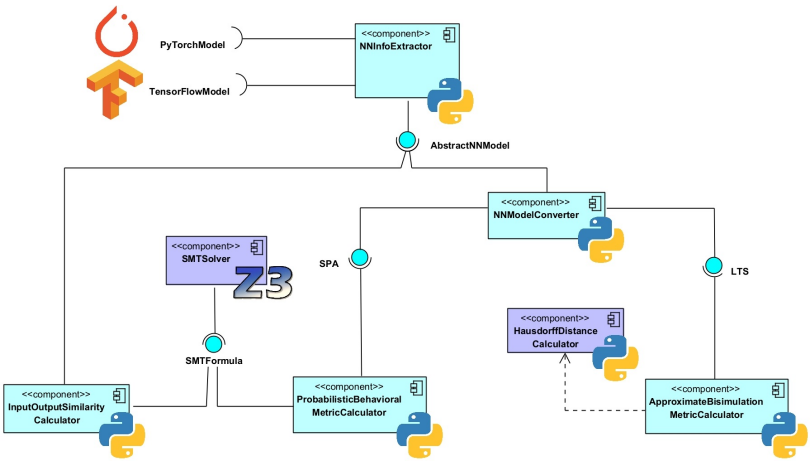

Figure 2.2: Component Diagram of the Developed Library's Static Architecture.

The *NNInfoExtractor*[1] component is the one responsible for extrapolating the details of interest of a given neural network coming from one of the supported frameworks, currently PyTorch and TensorFlow. In the current version, the information extracted by this component consists in the neural network architecture in terms of layers, nodes, weights, biases and activation functions used in each layer of the network. All those information are then used to build the *AbstractNNModel* object, which is the main standardized data structure used to represent neural networks in our library.

[1] The range of supported models strictly depends on this component: for any new framework that needs to be supported, a new extension inside the *NNInfoExtractor* must be implemented. The same can be said for other possible families of neural networks outside the typical multi-layer perceptron.

The *InputOutputSimilarityCalculator*[2] is one of the components receiving the *AbstractNNModel* object as input. The main functionalities of this component consist in calculating the similarity between the input-output relations of two neural networks according to the approach explained by Eleftheriadis et al. [10], providing a reliable measure of how much two neural networks are similar in terms of

[2] We consider this component as a utility in support in the evaluation of the other methodologies explored in this project. Even if the used methodology falls in the category of white-box approaches, it provides a way to compare neural networks based on inputs and outputs, namely without considering the internal intermediate states of neurons in the final decision of the similarity.

[10] Eleftheriadis et al. "On Neural Network Equivalence Checking Using SMT Solvers"

the input-output behavior through the use of SMT formula encoding. This module heavily relies on the external *SMTSolver* component, which is responsible for constructing and solving the SMT formulas generated by the similarity calculator. The SMT Solver used in this project is Z3 and it'll be further explained in Section 2.3.

The *NNModelConverter* is the other component that receives the *AbstractNNModel* object as input and has the purpose of converting the neural network model in other formats compatible with the remaining implemented white-box similarity algorithms. The module implements the possible criteria of conversion to Simple Probabilistic Automatas (SPA) and Labelled Transition Systems (LTS), which are also represented as internal data structures in the library.

The *ProbabilisticBehavioralMetricCalculator* implements the algorithm and calculates the metric proposed by Deng et al. [8] and Chen, Han, and Lu [6]. The component builds upon those definitions to provide an overall degree of similarity based on the specific computed distances of couples of states deriving from the neural network conversion. For this reason, as opposed to the *InputOutputSimilarityCalculator* module, we consider this methodology more sensible to the internal structure of the neural networks.

[8] Deng et al. "Metrics for Action-labelled Quantitative Transition Systems"
[6] Chen, Han, and Lu "On metrics for probabilistic systems: Definitions and algorithms"

The *ApproximateBisimulationMetricCalculator* relies and further adapts the concepts treated by Prabhakar [18], implementing a similarity measure based on the approximate bisimulation of transition systems derived from neural networks. Functionalities of the module include the calculation of a metric based on paths weight to establish approximate bisimilarity between nodes, and the calculation of approximate bisimilarity between whole transition systems, which leverages the state metric in combination with an adaptation of the Hausdorff's algorithm. For this reason the module relies on the represented external *HausdorffDistanceCalculator* component, which offers the functionalities for the calculation of the Hausdorff distance between two sets of points.

[18] Prabhakar "Bisimulations for Neural Network Reduction"

As for the *ProbabilisticBehavioralMetricCalculator*, we also consider this methodology tied to the internal structure of the neural networks rather than the simple input-output behavior.

The three components in the bottom side of the diagram provide the three different views on the similarity of the neural networks, which will be further discussed in Chapter 4.

2.3 TECHNOLOGIES

The implementation of our project relies on a combination of software frameworks, programming languages, and mathematical solvers to effectively model, analyze, and compute bisimulation metrics and pseudometrics. This section provides an overview of the key technologies used in the development process, highlighting their roles and contributions to different aspects of the system.

Python For this implementation, we used Python, a widely adopted programming language in scientific computing, machine learning, and deep learning. Python's popularity in these domains stems from its simplicity, readability, and extensive ecosystem of libraries that support numerical computation, data manipulation, and artificial intelligence. Specifically, Python offers powerful libraries such as NumPy and SciPy for mathematical operations, pandas for data handling, and Matplotlib for visualization.

Most importantly, frameworks like TensorFlow, PyTorch, and Scikit-learn make it an ideal choice for developing and experimenting with machine learning models. The strong community support, cross-platform compatibility, and seamless integration with other languages also contributed to make python the preferred choice for this project. The Z3 library, which we used for SMT solving, is a clear example of the power of Python in integrating with other languages.

Figure 2.3: Python's Logo.

Z3 To prove the satisfiability of SMT formulas we have used the Z3 solver, a state-of-the-art SMT solver developed by Microsoft Research. Z3 is a high-performance theorem prover that supports both linear and non-linear arithmetic, uninterpreted functions, bit-vectors, arrays, datatypes, and quantifiers. The solver is written in C++ and provides APIs for C, C++, Java, and Python. It is widely used in the formal verification community and has been employed in various research projects and industrial applications. The choice of this solver is motivated by its efficiency in handling both boolean logic and linear arithmetic efficiently, which are the main components of the formulas generated both by the module on SMT-based equivalence checking ("*InputOutputSimilarityCalculator*") and the module on the SPA behavioural distance ("*ProbabilisticBehavioralMetricCalculator*").

However, as generally true for SMT solvers, Z3 has limited support for non-linear arithmetic, a those problems are often undecidable in the general case. Unlike linear arithmetic, which can be efficiently handled using simplex-based algorithms, non-linear real arithmetic (NRA) and non-linear integer arithmetic (NIA) are much more complex and require more sophisticated algorithms like bounds propagation, cylindrical algebraic decomposition (CAD), and heuristic techniques. These methods may be extremely slow, as in the worst-case complexity of CAD that is doubly exponential, and do not guarantee termination for all possible non-linear formulas.

Figure 2.4: Z3's Library Logo.

PyTorch The primary framework used in our project is PyTorch, a widely used deep learning known for its flexibility, dynamic computation graph, and strong GPU acceleration. This model supports the development of various machine learning models, including feed-forward neural networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs), transformers, and generative models.

Models in PyTorch are typically built using the torch.nn.Module class, where layers and operations are defined as a structured compu-

Figure 2.5: PyTorch's Logo.

tation graph. Available layers include fully connected layers, convolutional layers, recurrent layers, normalization layers, and activation functions. We ensured our library is compatible with PyTorch models due to its widespread adoption in research and industry, ensuring existing deep learning pipelines based on the traditional multi-layer perceptron architecture can be easily integrated with our framework.

TensorFlow We also considered another leading deep learning framework that, like PyTorch, supports the development of a wide range of machine learning models. TensorFlow, much like the previously mentioned model, provides automatic differentiation, GPU acceleration, and a rich collection of prebuilt layers and re-activation functions. Also, its primary distinction lies in its static computation graph (Graph mode) through TensorFlow 1.x, which was later complemented by eager execution in TensorFlow 2.x, making it more similar to PyTorch's dynamic computation graph.

Model construction in TensorFlow is done using the tf.keras API, which provides a high-level abstraction similar to PyTorch's torch.nn.Module. Both frameworks support a variety of activation functions (ReLU, Sigmoid, Tanh, Softmax, etc.) and allow defining custom layers when needed. While PyTorch is often favored in research due to its intuitive debugging and dynamic execution, TensorFlow remains dominant in production environments thanks to tools like TensorFlow Serving and TensorFlow Lite, which facilitate model deployment on various platforms. Even if PyTorch models see a widespread use in academia and research, TensorFlow remains a particularly strong alternative in those scenarios where scalability and production-ready tools are key considerations.

Figure 2.6: TensorFlow's Logo.

NumPy NumPy is a fundamental library for numerical computing in Python, providing efficient array operations, linear algebra functions, and support for large multi-dimensional datasets.

In this project, NumPy has been employed in the neural network conversion process, as it permits to define custom probability distributions as truncated normal distributions. Those distributions are used to provide sample inputs for the construction of SPA models according to the statistical properties of the original datasets.

Figure 2.7: NumPy's Logo.

Pandas Pandas is a powerful Python library for data manipulation and analysis, offering flexible data structures such as DataFrames and Series for handling structured data efficiently. It provides a wide range of functionalities, including data cleaning, transformation, and aggregation, making it a staple for data science and research applications.

In this project, Pandas has been utilized for dataset manipulation, enabling the preprocessing and organization of data required to set up experimental trials. Its capabilities facilitated seamless data handling, ensuring consistency and accuracy in the experimental design.

Figure 2.8: Pandas' Logo.

3

Implementation

In this chapter we provide explanations on the algorithms and implementation choices adopted in the system, as well as the involved mathematical principles behind the proposed solutions.

The full library implementation can be found at `https://github.com/marco-caputo/neural-bisimulation.git`.

3.1 SMT EQUIVALENCE CHECKING

The implementation of the SMT-based equivalence checking has been carried out according to the proposed definitions from Eleftheriadis et al. [10]. The method firstly relies on the construction of SMT formulas that represent the input-output relation of a given neural network. From the abstract specification obtained from a TensorFlow or Pytorch model, we can derive a formula involving a number of variables depending on the dimension of the various layers of the network. A variable for each input x, affine transformation z (hidden neuron input), intermediate output h (hidden neuron output) and output y is defined, as depicted in the example in Figure 3.1. All those variables are linked by a set of constraints that represent the operations of the network, such as the affine transformation obtained by the weights and biases of a linear layer and the application of the activation function.

[10] Eleftheriadis et al. "On Neural Network Equivalence Checking Using SMT Solvers"

Indeed, considering a neural network with n inputs, the relation between the input of the network and the input z_i of each i-th neuron in the first hidden layer can be encoded in the following general formula:

$$\bigwedge_{i=1}^{n} z_i = \sum_{j=1}^{n} w_{ji}^{(1)} x_j + b_i^{(1)}$$

where $w_{ji}^{(1)}$ and $b_i^{(1)}$ are the weight from the j-th input to the i-th neuron in the first layer and the bias of the i-th neuron in the first layer, respectively.

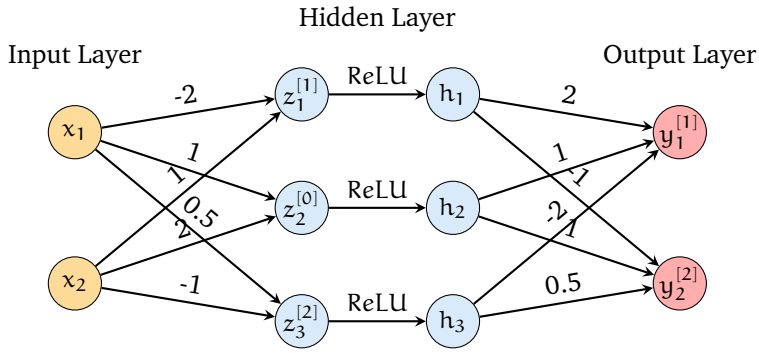

Figure 3.1: Example of a Feed-Forward Neural Network.

The model depicted is made of an input layer of 2 inputs (x_1, x_2), an output layer with 2 outputs (y_1, y_2), no activation function and 1 hidden layer with 3 neurons and a ReLU activation function. The values on the transitions correspond to the weights and the upperscript values to the biases.

Considering the example in Figure 3.1, for instance, the formula expressing the input of the hidden layer can be written as:

$$(z_1 = -2x_1 + x_2 + 1) \wedge (z_2 = x_1 + 2x_2) \wedge (z_3 = \frac{1}{2}x_1 - x_2 + 2)$$

Each different activation function can then be encoded in the corresponding formula, which is then added to the general formula as a constraint on the value of the output of the neuron. If we consider the ReLU activation function, as in the example, the formula that encodes the relation between the neuron inputs z_i and the outputs h_i in a hidden layer of n neurons can be written, in general, as:

$$\bigwedge_{i=1}^{n} (z_j \geqslant 0 \wedge h_j = z_j) \vee (z_j < 0 \wedge h_j = 0)$$

Once a comprehensive formula $\varphi(\bar{x}, \bar{y})$ that represents the relation between the input variable $\bar{x}$ and the output variable $\bar{y}$ of the whole network has been constructed, the equivalence checking with another network encoded into a formula $\varphi'(\bar{x}', \bar{y}')$ can be performed by constructing a new formula Φ which is the conjunction of the two formulas $\varphi(\bar{x}, \bar{y})$ and $\varphi'(\bar{x}', \bar{y}')$, combined with inputs and outputs constraints. While the inputs constraints consist in strict equivalence, the output constraints depend on the desired level of equivalence to be checked between the two networks. Of course, the set of inputs I and outputs O must be the same for both networks, so the general formula representing the non-equivalence according to a certain equivalence eq, in the closest form to implementation, can be written as:

$$\Phi_{eq} = \exists \bar{x}, \bar{x}' \in I, \bar{x} = \bar{x}' \wedge \varphi(\bar{x}, \bar{y}) \wedge \varphi(\bar{x}', \bar{y}') \neg eq(\bar{y}, \bar{y}') \tag{3.1}$$

Thanks to the negation of the output constraints, by solving the formula Φ with an SMT solver it is possible to determine if the two networks are equivalent whenever the formula is unsatisfiable. In the case of satisfiability, a counterexample can be extracted from the

model returned by the solver, which can be used to identify the input values that cause the two networks to produce different outputs.

The different output constraints determine the type of equivalence that is checked between the two networks. Let m be the number of outputs of the network, the definitions of the different types of equivalences available in our implementation[1] are the following:

- **Strict Equivalence**[2]: the output of the two networks must be exactly the same for all possible inputs. The formula that represents this type of equivalence is:

$$\mathrm{eq}(\bar{y}, \bar{y}') := \bigwedge_{i=1}^{m} y_i = y'_i$$

- **(p, ϵ)-approximate Equivalence**[3]: the output of the two networks must be at most ϵ far from each other according to the p-norm. The formula that represents this type of equivalence is:

$$\mathrm{eq}(\bar{y}, \bar{y}') := \left(\sum_{i=1}^{m} |y_i - y'_i| \right)^{1/p} \leqslant \epsilon$$

- **argmax Equivalence**: the output of the two networks must have the highest value in the same position. The formula that represents this type of equivalence is:

$$\mathrm{eq}(\bar{y}, \bar{y}') := \mathrm{argmaxis}(\bar{y}, i, m) \wedge \mathrm{argmaxis}(\bar{y}', i', m) \wedge i = i'$$

  where $\mathrm{argmaxis}(\bar{y}, i, m)$ is a formula that represents the fact that the i-th element of the vector $\bar{y}$ is the maximum element of the vector, and is defined as:

$$\mathrm{argmaxis}(\bar{y}, i, m) := \left(\bigwedge_{j=1}^{i-1} (y_i > y_j) \right) \wedge \left(\bigwedge_{j=i+1}^{m} (y_i \geqslant y_j) \right)$$

  This is a useful type of equivalence to check when the involved neural networks perform multi-classification tasks, typically such that the output is a vector of probabilities for each class. So the equivalence is supposed to verify weather two models assign the highest probability to the same class for each input. This is the same as checking if both neural networks predict the same class for any possible input.

Limitations For reasons tied to the general applicability of SMT solvers, we consider the application of the SMT equivalence checking of neural networks limited to the case of linear activation functions, in general. Linear versions of the most used activation functions, commonly known as "hard" versions, have been implemented to allow the approximate verification of a wider range of neural networks. Examples of those functions are the HardSigmoid, HardTanh, and

[1] All the formulas presented are written in the positive form, while the implementation uses the explicit negation of the output constraints for performance reasons.

[2] Being the strictest form of equivalence, its implementation takes into consideration the possibility of using a tolerance ϵ to account for the numerical errors introduced by the floating-point arithmetic

[3] The p-norm of an output y is defined as $\|y\|_p = \left(\sum_{i=1}^{m} |v_i|^p\right)^{1/p}$, and that the norm is also defined for $p = \infty$ as $\|y\|_\infty = \max_{i=1}^{m} |v_i|$, which corresponds to the maximum absolute difference between the two outputs. In our implementation, the (p, ϵ)-approximate equivalence is implemented for every value of p, with the case $p = \infty$ corresponding to the case of strict equivalence with tolerance ϵ.

HardSwish functions, which are piecewise linear approximations of the Sigmoid, Tanh, and Swish functions, respectively.

[3] Albarghouthi "Introduction to Neural Network Verification"

However, another solution to the problem proposed by Albarghouthi [3], which is also well-suited for our implementation, is the *over-approximation* of non-linear monotonic functions behavior, like Tanh and Sigmoid. Assuming f is a monotonically increasing function with a lower bound lb and an upper bound ub of its codomain, it is possible to pick a sequence of n input values $c_1 < c_2 < \ldots < c_n$ to encode the function input-output relation as:

$$
\begin{aligned}
&(x \leqslant c_1 \Rightarrow lb < f(x) \leqslant f(c_1)) \wedge \\
&(c_1 < x < c_2 \Rightarrow f(c_1) \leqslant f(x) < f(c_2)) \wedge \\
&\vdots \\
&(c_n < x \Rightarrow f(c_n) < f(x) \leqslant ub)
\end{aligned}
$$

A similar approach has been adopted in our implementation in order to provide an over-approximation of the Sigmoid function, as it plays a crucial role in the algorithm of the SMT-based approximate equivalence metric. In particular, we decided to represent the Sigmoid function as a formula which relies on a set of points in the input domain, which are used to approximate the function behavior. However, those points are evenly distributed with respect to the corresponding output values, in order to provide an interval of reference for each hundredth value of the output. Namely, let σ be the Sigmoid function, let $\sigma^{-1}(y) = \ln(y/(1-y))$ be the pre-computed value of the inverse function of σ for $y \in (0, 1)$, we can define the correspondent SMT formula with a precision on the output value of 0.01 as:

$$
\begin{aligned}
\sigma(x) = {} & (x \leqslant \sigma^{-1}(0.05) \Rightarrow 0.0) \wedge \\
& (\sigma^{-1}(0.005) < x \leqslant \sigma^{-1}(0.015) \Rightarrow 0.01) \wedge \\
& \vdots \\
& (\sigma^{-1}(0.985) < x \leqslant \sigma^{-1}(0.995) \Rightarrow 0.99) \wedge \\
& (\sigma^{-1}(0.995) < x \Rightarrow 1.0)
\end{aligned}
$$

This approach allows us to provide a more accurate approximation of the function with respect to its usage in the context of neural networks, by defining piecewise-constant values for different input ranges and making the approximation able to produce any percentage of confidence in the output value. However, this implementation increases the overall complexity of the SMT formula associated with the neural network, which may lead to a higher computational costs.

For this reason we decided to substitute instances of the Sigmoid function with the Hard Sigmoid function, the piecewise linear approximation of the Sigmoid function, which is instead represented by the following formula:

$$\begin{aligned}\sigma(x) = (x \leqslant -3 \Rightarrow 0) \wedge \\ (x \geqslant 3 \Rightarrow 1) \wedge \\ \left(-3 < x < 3 \Rightarrow \frac{x+3}{6}\right)\end{aligned}$$

Approximate Equivalence Metric We now provide a brief overview of the approximate equivalence metric we designed for neural networks on top of the SMT-based approach. The metric is based on the idea of comparing the output of two neural networks on a range of input values and checking whether the two outputs do not deviate more than a certain threshold according to the given p-norm, and finding the minimum threshold that satisfies the condition. As previously mentioned, this is equivalent to finding the minimum value of ϵ such that the two neural networks are (p, ϵ)-approximate equivalent according to some p. We consider the particular case of binary classifiers having a unique output neuron equipped with a Sigmoid activation function, and we define the approximate equivalence metric as follows:

Definition 11 (Approximate Equivalence Metric)**.** Given two neural networks $N_1 : X \to [0, 1]$, $N_2 : X \to [0, 1]$ defined on the same set of inputs X and having a unique output neuron with a Sigmoid activation function. The approximate equivalence distance ϵ^* between N_1 and N_2 is defined as:

$$\epsilon^* = \max_{x \in X} |N_1(x) - N_2(x)|$$

As previously mentioned, the metric can be equivalently defined in terms of (p, ϵ)-approximate equivalence as follows:

$$\epsilon^* = \min\{\epsilon : |N_1(x) - N_2(x)| \leqslant \epsilon \forall x \in X\} \tag{3.2}$$

Proposition 12. *The approximate equivalence metric is a metric on the space of single-output neural networks with the same input domain.*

The proposition can be easily proven by showing that the approximate equivalence metric satisfies the properties of a metric. In fact, let X be any input domain, the metric satisfies the property of non-negativity, since the value of the metric is always obtained as an absolute value; the property of identity of indiscernibles, since $|N(x) - N(x)|$ for all $x \in X$ is always equal to 0; the property of symmetry, because $|N_1(x) - N_2(x)| = |N_2(x) - N_1(x)|$ for all $x \in X$, so the set of ϵ values for both the distances is the same, and so the maximum ϵ^*. Finally, to prove the triangle inequality, we can consider three neural networks N_1, N_2 and N_3. First we observe that, since the optimal $x \in X$ that maximizes the output divergence of two networks might not be the same for both, we have:

$$\max_{x \in X} |N_1(x) - N_2(x)| + \max_{y \in X} |N_2(y) - N_3(y)| \geqslant \max_{x \in X} (|N_1(x) - N_2(x)| + |N_2(x) - N_3(x)|) \tag{3.3}$$

Let $x^* \in X$ be one of the inputs that maximizes the second term of the inequality above, we now examine the quantity:

$$\max_{x \in X} (|N_1(x) - N_2(x)| + |N_2(x) - N_3(x)|) \;=\; |N_1(x^*) - N_2(x^*)| + |N_2(x^*) - N_3(x^*)| \;=\; \epsilon^*$$

Due to the presence of the double absolute value, we consider in Equation 3.4 all possible cases of the relative ordering of values $N_1(x^*)$, $N_2(x^*)$ and $N_3(x^*)$. For each possible case we have that $\epsilon^* \geqslant |N_1(x^*) - N_3(x^*)| = \max_{x \in X} |N_1(x^*) - N_3(x^*)|$. This, combined with the Equation 3.3, is sufficient to we conclude the triangle inequality holds.

We now propose an algorithm to compute the approximate equivalence metric between two neural networks. This algorithm leverages the SMT-based approach for encoding the neural networks and relies on the alternative definition of the metric as described in 3.2. However, since a single value of ϵ can be used to determine the (p, ϵ)-approximate equivalence of the two networks though a SMT formula, the algorithm performs a binary search over the possible values of ϵ. This is done by iteratively checking the satisfiability of the SMT formula for the given ϵ value, and adjusting the search interval accordingly. The number of iterations is logarithmic in the size of the interval, and the algorithm terminates when the interval is smaller than a given threshold. This means that the algorithm can compute the real value of the metric with an arbitrary precision δ in a finite number of steps, to the expense of higher computation times. This behavior is described in Algorithms 1 and 2.

```
Require: Neural Networks N_1, N_2 with domain X and
         codomain [0, 1]
Require: Input bounds B on X
Require: Precision δ > 0
l ← 0;
u ← 1;
for i ← 1 to ⌈log_2(1/δ)⌉ do
    ε ← (l+u)/2;
    if AreApproximateEquivalent(N_1, N_2, B, ε) then
        u ← ε;
    end
    else
        l ← ε;
    end
end
return (l+u)/2
```

Algorithm 1: Approximate Equivalence Metric algorithm.

Note: the p value does not affect the algorithm, as the various p-norms are equivalent for the case of a single output neuron.

Since we are considering the specific case of binary classifiers equipped with a Sigmoid activation function on their output neuron, the algorithm is designed to work with an interval of ϵ values in the range $[0, 1]$, which extremes are the minimum and maximum

possible output values of the networks, along with the minimum and maximum values of the metric.

$$
\begin{aligned}
&\text{(i):} \quad N_1(x^*) \leqslant N_2(x^*) \leqslant N_3(x^*) \\
&\Rightarrow \begin{cases} aN_1(x^*) - N_2(x^*) \leqslant 0 \\ N_2(x^*) - N_3(x^*) \leqslant 0 \\ N_1(x^*) \leqslant N_3(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* = -N_1(x^*) \cancel{+N_2(x^*)} \\ \qquad \cancel{-N_2(x^*)} + N_3(x^*) \\ N_3(x^*) - N_1(x^*) \leqslant 0 \end{cases} \\
&\Rightarrow \ \epsilon^* = |N_1(x^*) - N_3(x^*)| \\
&\text{(ii):} \quad N_1(x^*) \leqslant N_3(x^*) \leqslant N_2(x^*) \\
&\Rightarrow \begin{cases} N_1(x^*) - N_2(x^*) \leqslant 0 \\ N_2(x^*) - N_3(x^*) \geqslant 0 \\ N_1(x^*) \leqslant N_3(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* = -N_1(x^*) + N_2(x^*) \\ \qquad +N_2(x^*) - N_3(x^*) \\ N_2(x^*) \geqslant N_3(x^*) \\ N_1(x^*) \leqslant N_3(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* \geqslant -N_1(x^*) + N_3(x^*) \\ \qquad \cancel{+N_3(x^*)} \cancel{-N_3(x^*)} \\ N_3(x^*) - N_1(x^*) \geqslant 0 \end{cases} \\
&\Rightarrow \ \epsilon^* = |N_1(x^*) - N_3(x^*)| \\
&\text{(iii):} \quad N_3(x^*) \leqslant N_1(x^*) \leqslant N_2(x^*) \\
&\Rightarrow \begin{cases} N_1(x^*) - N_2(x^*) \leqslant 0 \\ N_2(x^*) - N_3(x^*) \geqslant 0 \\ N_3(x^*) \leqslant N_1(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* = -N_1(x^*) + N_2(x^*) \\ \qquad +N_2(x^*) - N_3(x^*) \\ N_2(x^*) \geqslant N_1(x^*) \\ N_3(x^*) \leqslant N_1(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* \geqslant \cancel{-N_1(x^*)} \cancel{+N_1(x^*)} \\ \qquad +N_1(x^*) - N_3(x^*) \\ N_1(x^*) - N_3(x^*) \geqslant 0 \end{cases} \\
&\Rightarrow \ \epsilon^* = |N_1(x^*) - N_3(x^*)| \\
&\text{(iv):} \quad N_2(x^*) \leqslant N_1(x^*) \leqslant N_3(x^*) \\
&\Rightarrow \begin{cases} N_1(x^*) - N_2(x^*) \geqslant 0 \\ N_2(x^*) - N_3(x^*) \leqslant 0 \\ N_3(x^*) \leqslant N_1(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* = N_1(x^*) - N_2(x^*) \\ \qquad -N_2(x^*) + N_3(x^*) \\ N_3(x^*) \geqslant N_2(x^*) \\ N_3(x^*) \leqslant N_1(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* \geqslant N_1(x^*) - N_3(x^*) \\ \qquad \cancel{-N_3(x^*)} \cancel{+N_3(x^*)} \\ N_1(x^*) - N_3(x^*) \geqslant 0 \end{cases} \\
&\Rightarrow \ \epsilon^* \geqslant |N_1(x^*) - N_3(x^*)| \\
&\text{(v):} \quad N_2(x^*) \leqslant N_3(x^*) \leqslant N_1(x^*) \\
&\Rightarrow \begin{cases} N_1(x^*) - N_2(x^*) \geqslant 0 \\ N_2(x^*) - N_3(x^*) \leqslant 0 \\ N_1(x^*) \leqslant N_3(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* = N_1(x^*) - N_2(x^*) \\ \qquad -N_2(x^*) + N_3(x^*) \\ N_1(x^*) \geqslant N_2(x^*) \\ N_1(x^*) \leqslant N_3(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* \geqslant \cancel{N_1(x^*)} \cancel{-N_1(x^*)} \\ \qquad -N_1(x^*) + N_3(x^*) \\ N_1(x^*) - N_3(x^*) \leqslant 0 \end{cases} \\
&\Rightarrow \ \epsilon^* \geqslant |N_1(x^*) - N_3(x^*)| \\
&\text{(vi):} \quad N_3(x^*) \leqslant N_2(x^*) \leqslant N_1(x^*) \\
&\Rightarrow \begin{cases} N_1(x^*) - N_2(x^*) \geqslant 0 \\ N_2(x^*) - N_3(x^*) \geqslant 0 \\ N_3(x^*) \leqslant N_1(x^*) \end{cases} \quad \Rightarrow \begin{cases} \epsilon^* = N_1(x^*) \cancel{-N_2(x^*)} \\ \qquad \cancel{+N_2(x^*)} - N_3(x^*) \\ N_1(x^*) - N_3(x^*) \geqslant 0 \end{cases} \\
&\Rightarrow \ \epsilon^* = |N_1(x^*) - N_3(x^*)|
\end{aligned}
\tag{3.4}
$$

A parameter B is used for adding constraints for the range of input values considered in the definition of the SMT formula. This can be used to define the specific domain of the inputs: in case of binary or categorical inputs, the bounds can be set to restrict the input space to the possible discrete values.

The input bounds also emerged as a useful parameter to regulate the sensibility of the metric for continuous and unrestricted inputs. Indeed, allowing large bounds far beyond the actual range of typical non-degenerate inputs can lead to a significant increase in the overall distance between the two networks considered, which could fail to represent their actual behavior. In this sense, we used the parameter B to specify such range of input values and to restrict the search space of the algorithm to a more meaningful domain. This parameter should be set according to the distribution of the input values, so should take into consideration the mean and the standard deviation in order to provide a meaningful representation of the input space.

Require: Neural Networks N_1, N_2, input bounds B, threshold ϵ
Ensure: **True** if N_1 and N_2 are ϵ-approximate equivalent, else **False**
Let f_1 be the SMT formula encoding N_1 with bounds B;
Let f_2 be the SMT formula encoding N_2 with bounds B;
$f \leftarrow f_1 \wedge f_2 \wedge (|N_1(x) - N_2(x)| \geqslant \epsilon)$;
if f *is satisfiable for some* $x \in X$ **then**
  return False
end
else
  return True
end

Algorithm 2: Input-output Approximate Equivalence Checking algorithm.

3.2 NEURAL NETWORK CONVERSION

The conversion procedure of a neural network to other models of reference, such as probabilistic automatons and finite state processes, is a crucial step in the project, as it allows to provide a definition and a formal representation of the network's behavior to apply the bisimilarity metrics.

While existing works have proposed probabilistic automatas or deterministic finite automatas (DFA) as a suitable model for representing the change in state of recurrent neural networks (RNN) during training [21], [22], in order to produce a model which hopefully captured the semantics of classification tasks, we decided to adopt an interpretation based on the flow of the network outputs through the different layers of the network. During the inference phase, the network can be seen as a finite state process, where states are organized by layers and each represents a different configuration of outputs in such layer. So, let $\mathcal{S}_i$ be the set of states associated to layer i, we say

[21] Wang, Lawrence, and Niepert, "State-Regularized Recurrent Neural Networks to Extract Automata and Explain Predictions"

[22] Weiss, Goldberg, and Yahav, "Extracting automata from recurrent neural networks using queries and counterexamples"

that at the moment i in time the neural network will be in a state $s \in \mathcal{S}_i$, which specifies a specific property of outputs $h_{i,1}, h_{i,2}, \ldots, h_{i,n}$ of the involved n neurons in layer i. The properties characterizing different states can be defined in order to obtain an arbitrary degree of granularity regarding the configuration of outputs of each given layer.

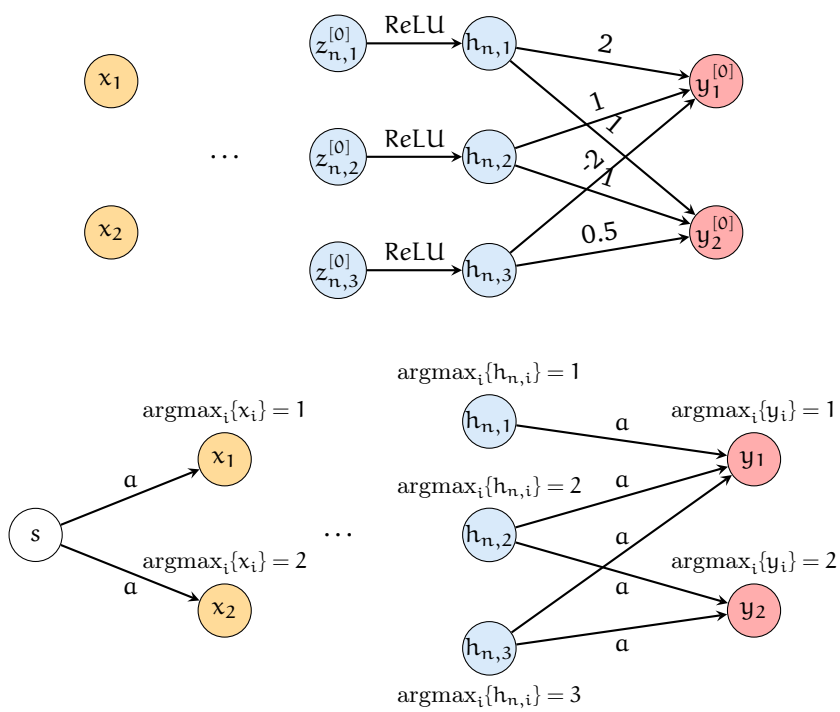

Figure 3.2: Example of argmax modeling of a MPL Neural Network with n hidden states as a Finite State Process.

As Figure 3.2 shows, the property we decided to adopt in this preliminary implementation is the *argmax* of the outputs of each layer, that is to say the index of the output with the highest value. This is a simple and effective way to capture the most relevant information regarding the network's behavior for classification tasks: by iteratively transitioning from one state to another, the finite state process will end up in a final state which uniquely identifies the class predicted by the network. In this way, the number of states in the finite state process is exactly equal to the number of neurons in the original network, while the possible actions are restricted to a singleton set $\{a\}$, representing the transition between states associated with a forward pass. So starting from the initial properties of the input data, the network will transition through states associated with all the different layers until it reaches the final state, which will be associated with the predicted class. We can also add an abstract initial state s, which represents the initial configuration of the network before receiving an input. From such representation, we can state that the presence of a transition between two states[4] is determined by the properties of possible inputs, the network's architecture and weights.

[4] Not all transitions between states from one layer to the next one could be present, as weights and input constraints could prevent the network from transitioning between certain couples of states.

Unveiling those patterns is exactly the purpose of one of the implemented conversion procedures which, in this case, relies on the Z3 SMT solver to determine the possibility of a transition between two states of interest. Using this approach, each candidate transition is encoded as a set of constraints involving the inputs as free variables, which are then passed to the solver to determine the satisfiability of the constraints. A transition is added to the finite state process if there exists at least one input which satisfies the constraints.

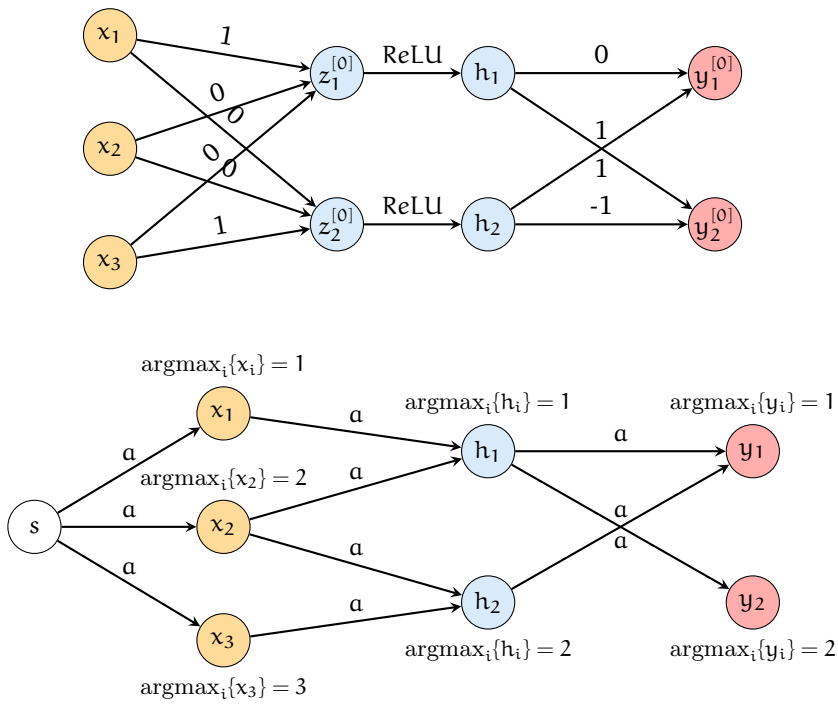

Figure 3.3: Example of SMT-based conversion of a Neural Network into a Finite State Process.

In this example, the included transitions are only those which are possible whenever inputs are non-negative.For instance, the transition from state x_1 to state $h_{n,1}$ is possible, as the following formula is satisfiable:

$(x_1 \geqslant 0 \wedge x_2 \geqslant 0 \wedge x_3 \geqslant 0) \wedge$
$(h_1 = \text{ReLU}(x_1) \wedge h_2 = \text{ReLU}(x_2)) \wedge$
$(x_1 > x_2 \wedge x_1 > x_3) \wedge$
$h_1 > h_2$

A simple example of the SMT-based conversion procedure is shown in Figure 3.3, where a neural network with one hidden layer is converted into a finite state process having a subset of all the possible transitions.

However, as can be observed, the approach does not precisely capture the full behavior of the network, as it does not take into account the likelihood of the transitions between states, but it limits itself to determining the existence of a possible input which allows the network to make that transition. For this reason we implemented a more precise, yet heuristic, conversion procedure which is based on the estimation of the probability of a transition between two states by sampling the input space. A random number generator is used to generate a large set of random inputs, which are then passed to the network to observe the transitions between states. The overall probability of a transition $s_1 \rightarrow s_2$ is then estimated as the ratio of the number of successful transitions from s_1 to s_2 over the total number of outgoing transitions from s_1. On the other hand,

the probabilities[5] associated to transitions $s \to x_i$, with s being the initial state and x_i being the state where the network receives an input having i as the argmax, are manually set as equidistributed, assuming no correlation between the input features. Since multi-layer perceptrons are usually trained using a scaled version of the input data, forcing the inputs to have average zero and unit variance, the random inputs can be generated by sampling from a normal distribution with such parameters. Moreover, additional bounding constraints can be added to the inputs, in order to limit the range of possible values and to avoid the generation of inputs which are not compatible with the real-world domain.

[5] These could be adjusted in a real-world scenario, where the general relation between input features is known.

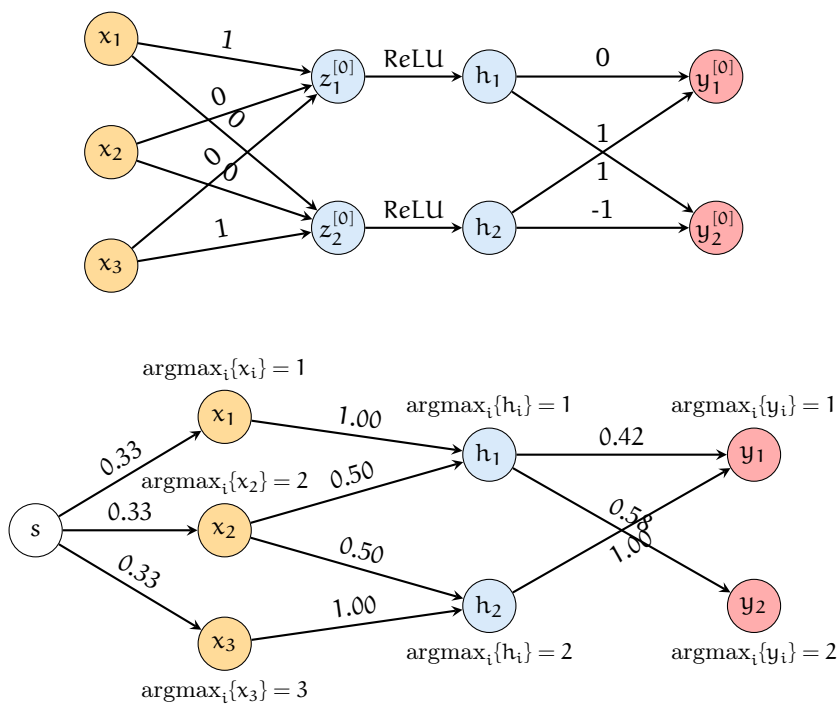

Figure 3.4: Example of conversion of a Neural Network into a Simple Probabilistic Automata obtained through input space sampling.

Another simple example of the probabilistic conversion procedure is shown in Figure 3.4, where the same neural network is converted into a simple probabilistic automata, obtained from a set of ten thousand random non-negative inputs drawn from a truncated normal distribution. As can be seen, many of the nuances regarding the possible transitions which could not be observed through the SMT based approach are now observable.

3.3 SPA BEHAVIORAL DISTANCE ALGORITHM

Similarly to the previous subsection, the algorithm proposed by [6] has undergone an adaptation and an implementation in Python, leveraging the Z3 SMT solver for proving the mathematical constraints arising from the behavioral pseudo-metric computations.

[6] Chen, Han, and Lu, "On metrics for probabilistic systems: Definitions and algorithms"

The core idea behind this part is to encode the computation of the bisimulation pseudo-metric as a set of SMT constraints, which are

then processed using the Z3 solver. The SPA structure is implemented as a dictionary of dictionaries of lists of dictionaries, capturing the state transitions and their associated probabilities.

The main logic of the algorithm, which is described in Algorithm 3, consists in proving the existence of a distance matrix d describing a possible pseudo-distance definition between the various states of the input automata, with the ultimate goal of finding the distance $d(s,t)$ between two specific states of interest s and t according to such distance matrix. The distance matrix is initialized at the beginning as a symbolic variable, representing the pseudo-metric distances between states. Then a binary search is performed, imposing an upper bound m on the distance between the two states $d(s,t)$ at each iteration. If the logical constraints on the distance matrix are satisfiable, the search is continued looking at lower values of m; otherwise, the search is continued with higher values of m. The number of iterations performed by the algorithm depends on the arbitrary precision described by the ϵ parameter, which is used to determine the width of the range of possible distance values which is returned as final result.

Require: A simple probabilistic automata $\mathscr{P} = (S, A, \rightarrow)$, and a rational value ϵ as tolerance, two states s, t in S.
Ensure: An interval $[\ell, u]$ such that $u - \ell \leqslant 2\epsilon$ and $d(s,t) \in [\ell, u]$
$\ell := 0, u := 1, m := \frac{1}{2}$;
for $\lceil \log(\frac{1}{\epsilon}) \rceil$ *steps* **do**
  if $\exists d.\overline{\text{postfixpoint}}(d) \wedge \text{pseudo}(d) \wedge d(s,t) \leqslant m$ **then**
    $u := m, m := \frac{l+u}{2}$;
  end
  else
    $\ell := m, m := \frac{l+u}{2}$;
  end
end
return $[\ell, u]$

Algorithm 3: Distance Approximation Algorithm, [6]

A first attempt in implementing the algorithm followed the exact way the algorithm is introduced in the paper and replicated all the different conditions and definitions. However, this approach revealed to be unfeasible due to the complexity of the produced formulas. Indeed, the Z3 solver is not generally able to handle constraints involving numerous existential or universal quantifiers, which are defined by the algorithm in the original form.

A first analysis of the algorithm led to the conclusion that many of the constraints could be reinterpreted as optimization problems and relaxed to a simpler form. However, this was not sufficient to make the algorithm compatible with the Z3 solver, which still failed to produce satisfiable results for the constraints. Moreover, the approach remained unfeasible in practice, as it included the resolution of many nested optimization problems, which lack a direct support in Z3 and

currently remain a challenging task for available tools.

These reasons led to the decision to simplify the algorithm by reducing its domain of applicability to the set of deterministic simple probabilistic automatas (DSPA), hence excluding the possibility of having different possible probability distributions for the same states and actions in the automata. This restriction did not impact negatively the specific application of the algorithm in the project, as the SPA generated by the adopted conversion of neural networks to automata is deterministic by construction. Moreover,the adaptation represents a closer approximation to the original metric proposed by Deng et al. [8].

[8] Deng et al. "Metrics for Action-labelled Quantitative Transition Systems"

As can be seen from Equation 3.5, the constraint to be satisfied was reduced in complexity by the removal of two nested optimization problems, which depended on the existence of different probability distributions.

$$\exists d. \bigwedge_{s,t\in S} \exists \max_{(a\in A|\iota(s,a)\neq\emptyset)} \begin{pmatrix} \max & y \\ \text{s.t. } y = \hbar d(a,d,s,t) \\ y = \hbar d(a,d,t,s) \end{pmatrix}$$

where

$$\hbar d(a,d,s,t) = \cancel{\max_{\eta\in\ell(s,a)}} \cancel{\min_{\eta'\in\ell(t,a)}} \begin{pmatrix} \max_{\vec{x}} \sum_{s\in S} (\eta(s)-\eta'(s))\cdot x_s \\ \text{s.t. } \bigwedge_{s\in S} 0 \leqslant x_s \leqslant 1 \\ \bigwedge_{s,t\in S} x_s - x_t \leqslant d(s,t) \end{pmatrix} \tag{3.5}$$

Even if the final simplified constraint was not directly representable as a single one-level optimization problem, and the d matrix was still a symbolic variable to be determined by the solver, the overall problem could be represented as a quantifier free formula through the following transformations:

- The inner-most optimization problem, representing the constraints of the Hutchinson metric, was transformed into a set of linear constraints imposing the equality of the objective functions of both the primal and dual formulation, as shown in Equation 3.6. Thanks to the properties of duality, we avoided recurring to existential or universal quantifiers, nevertheless providing a symbolic variable representing the optimal value of the objective function.

$$\exists y. y = \hbar d(a,d,s,t)$$
$$\Longleftrightarrow$$
$$\exists \vec{x},\pi,\nu. \begin{pmatrix} & \sum_{s\in S}(\eta(s)-\eta'(s))\cdot x_s & = & \sum_{s,t\in S} d(s,t)\pi_{s,t} + \sum_{s\in S}\nu_s \\ \wedge & \bigwedge_{s\in S} 0 \leqslant x_s \leqslant 1 & \wedge & \bigwedge_{s\in S}\sum_{t\in S}(\pi_{s,t}-\pi_{t,s}) + \nu_s \geqslant (\eta(s)-\eta'(s)) \\ \wedge & \bigwedge_{s,t\in S} x_s - x_t \leqslant d(s,t) & \wedge & \bigwedge_{s,t\in S}\pi_{s,t} \geqslant 0 \quad \wedge \quad \bigwedge_{s\in S}\nu_s \geqslant 0 \end{pmatrix} \tag{3.6}$$

- The property of being a 1-bounded pseudometric was encoded producing both the formulas for satisfiability and unboudedness

of the underlying optimization problem, as shown in the following Equation 3.7.

$$\begin{pmatrix} \exists\, y \\ \text{s.t. } y = \hbar d(a,d,s,t) \\ \quad y = \hbar d(a,d,t,s) \\ \quad \forall z.(z = \hbar d(a,d,s,t) = \hbar d(a,d,t,s)) \implies y \geqslant z \end{pmatrix} \\ \iff \\ \begin{pmatrix} \hbar d(a,d,s,t) = \hbar d(a,d,t,s) \text{ is satisfiable} \\ \wedge \\ (y = \hbar d(a,d,s,t) = \hbar d(a,d,t,s) \wedge y > M) \\ \text{is not satisfiable for sufficiently large } M \end{pmatrix} \tag{3.7}$$

- The optimality among the possible actions was encoded by combining the formulas mentioned above in bigger constraints, determining the existence of a feasible solution for the optimization problem and the boundedness of the objective function. Indeed, if the maximum value of the objective function is unbounded for at least one action, the whole problem is unbounded, so an optimal solution does not exists. Otherwise, if there exists at least a feasible solution for an action, the problem is bounded and an optimal solution must exist.
- The final constraint ranging over all couples of states was then represented as a combination of the constraints mentioned above, leveraging the same properties of satisfiability and boundedness. The formulas were passed to the Z3 solver, which was able to determine the satisfiability of the constraints and the unboundedness as separate SMT problems.

3.4 SINGLE ACTION PFSP APPROXIMATE BISIMULATION

The implementation of the probabilistic approximate bisimulation has been created by studying the one proposed by Girard and Pappas [11], which covered the approximate equivalence between two transition systems by using the Hausdorff distance [16].

[11] Girard and Pappas "Approximation Metrics for Discrete and Continuous Systems"
[16] *MS Windows NT Kernel Description*

The implementation we've developed is provided by the module *ApproximateBisimulationMetricCalculator*, better defined in the Section 2.2 and it is based on the specifications provided by the paper formerly cited. The key difference resides in some adaptations that have been made in order for it to be applied to feed-forward neural networks represented by PFSPs.

The two main functionalities added to the algorithm are: computing the probabilities of each node and evaluating the approximate bisimulation.

Nodes Probabilities Computation

The computation of node probabilities is a fundamental step required for the approximate bisimulation evaluation and its key components,

used to establish an equivalence relationship[6], are the transition function Δ and the observation function Π.

This process is executed within the class constructor as part of the initialization phase; each node in the feed-forward neural network is assigned a probability set, which consists of pairs $(v, p_{reach}(v))$, with v a final node and $p_{reach}(v)$ the probability to reach that node, where:

- Final nodes are nodes that have themselves as the sole entry in their probability set, with a probability of 100% to reach themselves.
- Non-Final nodes derive their probability set based on their connections to successive nodes.

Recursive computation process To compute the probability set for each node, the algorithm performs an optimized depth-first search (DFS) recursion, discussed in Subsection 3.4 - "*Search Matrix and Optimizations*". The probability set of a non-final node is calculated by following these steps:

1. Traverse its outgoing transitions and retrieve the probability sets of its successor nodes.
2. Combine the aforementioned probability sets using the transition probabilities associated with each outgoing edge. Specifically, for each final node in a successor's probability set, its probability is multiplied by the probability of reaching that successor.
3. Sum the combined probabilities are across all successors and assign it to the current state's probability set.

Base case and recursion rollback The recursion terminates at final nodes, which return their probability set containing only themselves with a probability of 1.0. As the recursion unwinds, each node's probability set is computed step by step, ensuring that no node is processed before its successors have been resolved. If a node has multiple successors, the recursion may need to continue exploring deeper branches before completing the probability set computation for that node.

Probabilistic Approximate Bisimulation Computation

This functionality represents the core purpose of the algorithm and relies on a variant of the Hausdorff distance function to compute the distance between two probabilistic feed-forward neural networks. The Hausdorff distance computation has been optimized for pairs of corresponding candidate layers from the two networks under analysis.

Given two feed-forward neural networks without distinguishing labels, a strong bisimulation assessment leads to the following result:
$\forall l_1 \in Layers_1, l_2 \in Layers_2 \quad (l_{1i}) \approx (l_{1i})$

[6] Two states are considered equivalent if and only if:

$$\forall v \in V,\ \Delta(p_1, v) = \Delta(p_2, v)$$

With only exception being in the output (or final) states, which are considered equivalent if they have the same label.

Instead of applying the Hausdorff function to entire state sets, the comparison is performed at the equivalence class level. For each pair of layers, their distance is computed, and the final result is obtained by combining the results from each couple of layers.

Two key factors influence the comparison outcome, beyond the probability distribution:

- **Number of Output States:** If one network has output states absent in the other, the probability of reaching those states increases the overall Hausdorff distance.
- **Number of Layers:** When a network has fewer layers, its last layer is compared with the remaining layers of the deeper network. This approach enables the comparison of networks with different depths, assessing whether they can still be considered approximately bisimilar despite this structural discrepancy.

Search Matrix and Optimizations

To optimize performance and avoid redundant calculations, a structured search matrix[7] is employed for both probability computation and approximate bisimulation evaluation. This approach prevents repeated expensive searches and recalculations since it efficiently stores and retrieves probability sets, ensuring computational efficiency in two main operations:

1. **Probability computation:** Each time a new probability set is derived, it is stored in the matrix at the position corresponding to the evaluated node. This allows the algorithm to check whether a probability set has already been computed for a given node[8], eliminating the need for redundant calculations.
2. **Bisimulation evaluation:** Instead of performing costly recursive traversals through the transition function to locate nodes within the neural network, the algorithm leverages the triple indexing of the search matrix.

To retrieve a node's probability set, the matrix requires three indexing parameters: the neural network ID (which can be `0` or `1`), the layer ID and the node's unique name. By structuring lookups this way, the algorithm significantly reduces computational overhead, enabling an efficient evaluation of the approximate bisimulation between the two systems.

[7] Structured as a three-dimensional data structure of type `List[List[Dict[str, Dict[str, float]]]]`.

[8] When a node is encountered after it has already been computed, the precomputed probability set is retrieved directly from the matrix.

4

Results

In this chapter we present the results of the experimental trials conducted to evaluate the proposed metrics and algorithms. We first provide an overview of the experimental setup, followed by a detailed analysis of the results obtained from the application of the algorithms to various couples of neural network models. We then present the results[1] of the experiments conducted to evaluate the performance of the metrics in terms of expressiveness and capability to reflect the semantic similarities, with respect to the required execution time and scalability of the algorithms. The analysis focuses on identifying patterns in model divergence with respect to input features, neural network architecture, and training objectives. Finally, we discuss the implications of the results and provide insights into the potential applications of the proposed metric.

[1] All results are presented as heatmaps and distance matrices, visually representing the differences between the models based on the metric of reference.

4.1 EXPERIMENTAL SETUP

The experiments were conducted on a pool of specifically trained neural network models, all designed to solve binary classification tasks based on four input features, three of which are continuous and one categorical. The models vary in terms of architecture and training datasets, with the aim of evaluating the differences of proposed metrics in different scenarios.

Datasets For the experiments, we used datasets from *Kaggle*[2], an open platform providing datasets and resources for data scientists, analysts, and machine learning engineers.

[2] https://www.kaggle.com

We selected a set of datasets related to health conditions. The preprocessed datasets and sub-datasets derived from those that were used in the experiments are as follows:

- **Stroke**: a dataset on hearth stroke prediction from south-african

The label column indicates the occurrence of a stroke. The data from the original dataset was already cleaned of missing values and outliers.

patients. This contains $100,000$ entries with 4 of the original 18 features describing the health conditions of the patients. The 4 selected features include Total Cholesterol, Systolic Blood Pressure, Obesity Index, and Smoking Status.

- **Female** and **Male**: two sub-datasets of the Stroke dataset containing only the entries of female and male patients, respectively. These two datasets played the role of slightly different datasets with respect to the Stroke dataset, as they represent particular cases of the original dataset.

The label column indicates the diet quality. This entry was, indeed, already present as a feature in the original dataset.

- **Diet**: a sub-dataset of the Stroke dataset containing the entries of patients with either a healthy or unhealthy diet. This dataset played the role of a semantically different dataset with respect to the Stroke dataset which maintains the same input feature domain and meaning.
- **Inverted**: a conversion of the Stroke dataset, where the label column was inverted, i.e., a 1 in the label column indicates the absence of a stroke as opposed to its presence. This dataset played the role of the semantically opposite dataset with respect to the Stroke dataset in terms of input-output mapping.

The label column indicates the presence of depression. As for the Stroke dataset, the data was already cleaned of missing values and outliers.

- **Depression**: a dataset on depression prediction among college students. This dataset contains $27,900$ entries with 4 features of the original 16 describing demographical and lifestyle information of the students. The 4 selected features include Age, CGPA (Cumulative Grade Point Average), Study Hours per day and Gender.

Models Three classes of multi-layer perceptron architectures were considered:

- **2HL-3N**: a two-hidden-layer network with three neurons in each hidden layer;
- **2HL-5N**: a two-hidden-layer network with five neurons in each hidden layer;
- **4HL-3N**: a three-hidden-layer network with three neurons in each hidden layer.

For all of the considered architectures, four input neurons were present, one for each feature, and the activation function was set to ReLU for the hidden layers. The output layer varied according to the algorithm under evaluation. In the case of the SMT-based metric, the output layer was a single neuron with a sigmoid activation function, while for the Approximate Bisimilarity percentage, the output layer consisted of two neurons with a softmax activation function. All models were created and trained in Python using sequential models from Tensor Flow library. We used the Adam optimizer with a loss function of binary cross-entropy in case of the SMT-based metric, and categorical cross-entropy for the Approximate Bisimilarity percentage. During the training phase of the models, we used a batch size of 128 and 30 epochs, with a validation split of 0.2.

Computational Environment All experimental trials were conducted on a machine equipped with an Intel Core i5-6600 CPU and 16GB of RAM. All the phases of the experiments are replicable and are available in the GitHub repository of the project.

4.2 SMT APPROXIMATE EQUIVALENCE METRIC

This section provides an analysis of the results obtained from the application of the SMT-based metric described in Section 3.1. We present our results illustrating both the computed evaluations of the metric and the execution time of the required computation, including the time required to generate the SMT formulae and the time required to solve them.

The heatmap in Figure 4.1 illustrates the results of the SMT-based metric applied to the models trained on the Stroke dataset with all the considered architectures. Specifically the considered datasets are *Female*, *Male* and *Diet*. Each combination of models comparison is represented by a cell in the heatmap, which has the role of a distance matrix between all nine trained models. These distances have been computed with a precision of 0.01, and for this reason distances on the diagonal differ by zero by an error of 0.004.

As can be seen, the similarity of the dataset highly influences the results of the metric. Relative differences between models trained on the same gender subset remain relatively low. For example, different architectures trained on *female* show relatively low divergence (0.277, 0.371, 0.418), indicating that architectural changes can alter predictions within the same category in a minor way. This non-negligible divergence could represent a difference in the way models with different architectures learn from the data: increasing the number of neurons and hidden layers (e.g., 2HL-5N and 3HL-3N) generally results in greater divergence from simpler models. However, the effect is not strictly monotonic, suggesting that network topology alone does not determine prediction variance.

Distances between models trained on different halves of the dataset (*Female/Male*) tend to conform to those observed between models trained on the same dataset but with different architectures. This suggests that the difference in stroke indicators between males and females do not differ significantly among the considered features, which indeed are considered to be common risk factors for cardiovascular diseases.

However, the diet classification models exhibit higher relative distances with others (values around 0.449–0.457), maintaining substantial divergence from stroke models. This is expected, as the diet dataset is semantically different from the stroke dataset, despite sharing the same input feature domain. This validates that different predictive targets influence model outputs considerably. It is also worth noting that the distances between the models trained on the *Diet* tended to be lower than those between the models trained on the *Stroke* dataset. Further investigation would be required to under-

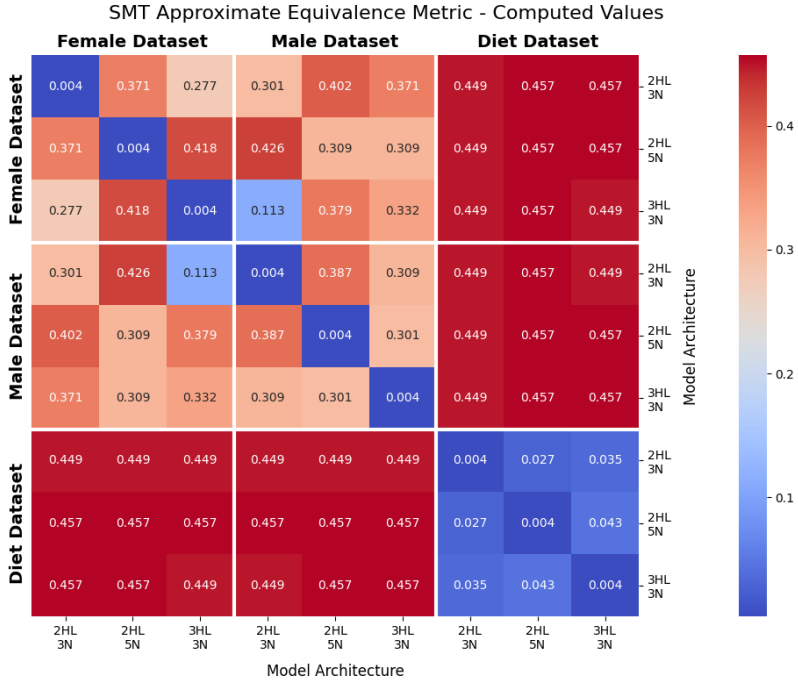

Figure 4.1: Results from SMT-Based Metric Applied to Models Trained on Variations of the *Stroke* Dataset.

stand the reasons behind this behavior.

The second heatmap in Figure 4.2 represents the execution times in rounded seconds required for computing the results shown in Figure 4.1. As can be seen, execution times can be considered demanding even for relatively simple models, with the time required to compute the metric rapidly increasing with the complexity of the models, with peaks reaching the order of half an hour for the most complex models. This is primarily due to the high amount of constraints that need to be generated and solved in the SMT solver, which increases with the number of neurons and layers in the models. For instance, models with architecture *2HL-5N* take significantly longer to be processed compared to *2HL-3N*, reflecting the particular influence on the computational burden introduced by increasing hidden neurons with respect to the number of layers.

The dataset-specific influence on the execution time is less evident: models trained separately on the *Male* and *Female* sub-datasets show some differences in execution time when they are compared with models trained on *Diet*, but these are relatively minor. As gender-based data segmentation does not drastically affect computational efficiency, no particular pattern can be found by correlating execution times with the computed outputs in Figure 4.1. While unsatisfiable formula usually take longer to be solved, the execution time does not appear to be correlated with the distance between the models, with identical models requiring similar times to be processed. This could be due to the particular formulae generated by the algorithm and the fortuitous possible simplifications that can be applied to them.

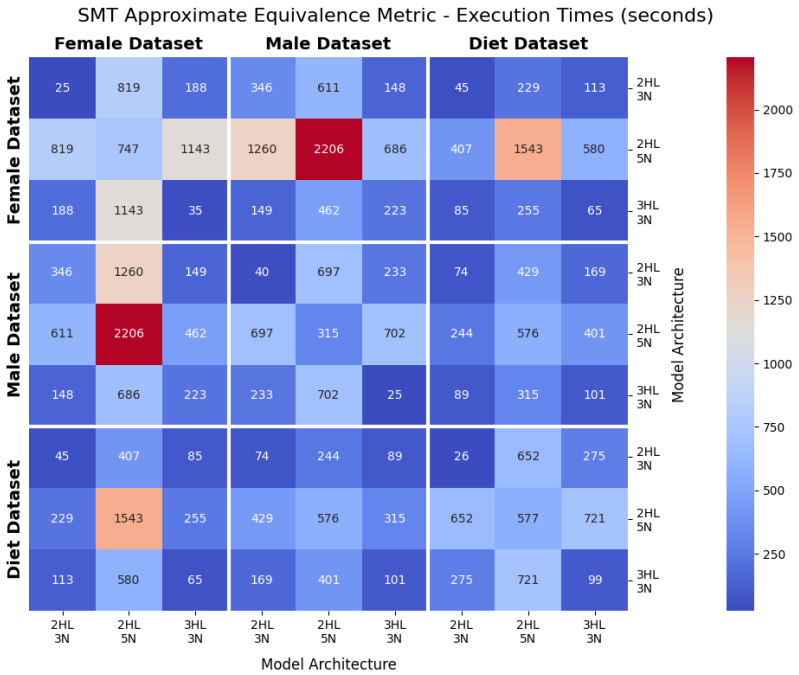

Figure 4.2: Execution Times from the SMT-Based Metric Applied to Models Trained on Variations of the *Stroke* Dataset.

The heatmap in Figure 4.3 illustrates the values of the SMT-based metric resulting from the comparison of the *Stroke* dataset models with other models with the same architecture but trained on dissimilar datasets.

Of course, the maximum closeness is observed between models having equal architecture and trained on the same *Stroke* dataset. However, the divergence between models trained on the *Stroke* dataset and those trained on the *Depression* dataset is relatively high, with values around 0.76. Considering the distances with the *Diet* dataset, we can observe that, according to the metric, the semantic difference between the two datasets *Stroke* and *Depression* is more pronounced, suggesting that completely changing the context of the learning task has a more significant impact on the model's predictions than changing the target variable within the same context.

A clear evidence of semantic dissimilarity can be observed in the comparison between the models trained on the *Stroke* dataset and the *Inverted* dataset, where the label column was inverted. This is coherent with the metric definition, as the inversion of the label column changes the output space of the model, leading to the most significant divergence in the predictions. Indeed the computed values are the highest possible, since models outputs are restricted in the range [0, 1], reflecting the fact that the *Inverted* dataset is semantically opposite to the *Stroke* dataset.

Another important aspect emerging from the heatmap is the fact that the divergence between couples of models trained on different datasets tended to stick to the same order of magnitude when those couples of models have common architecture. This suggests that the

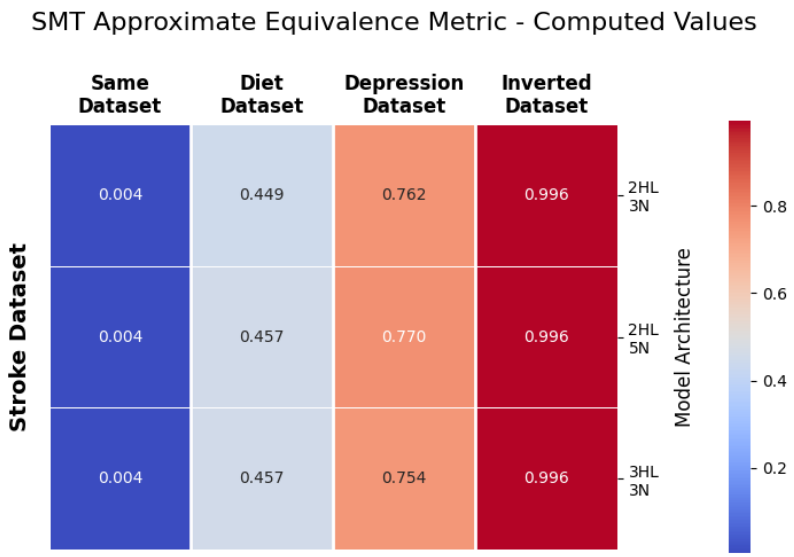

Figure 4.3: Results from SMT-Based Metric Applied to Models Trained on Dissimilar Dataset.

architecture of the neural network has can have a significant impact on the divergence of the models.

4.3 APPROXIMATE BISIMULATION DISTANCE

This section provides an analysis of the results obtained from the application of the Approximate Bisimulation algorithm described in Section 3.4. We provide results illustrating both the computed evaluations of the metric in the two possible forms, hence maximum distance between layers (**Max**) and average distance between layers (**Avg**). As for the previous section, we also provide execution times, which include the time required to make the conversion of both considered models into the corresponding PFSPs and the time required to compute the distance between the two PFSPs.

The heatmaps in Figures 4.4 and 4.5 illustrate the results of the Approximate Bisimulation metric applied to the pool of nine models of different dimensions trained on the variations of the *Stroke* dataset. The metric is computed in both forms, **Max** and **Avg**, and the results are shown in two separate heatmaps. This time, the main diagonal of the heatmaps is exactly zero, as the algorithm is capable of computing the exact distance between the two models, given a possible floating-point error. As can be immediately noticed, the results of the two forms follow a similar pattern, with the **Max** form providing higher values than the **Avg** form, as expected.

The **Max** proves itself to be generally more sensitive to variations in the produced PFSP models, often providing a value close to 1 – the maximum possible value – for couples of models trained on different tasks, e.g. those predicting strokes and those predicting diet quality. Distances of 1 are also observed between models trained on the same dataset but with architectures differing in the number of layers, suggesting that the algorithm is more susceptible to changes

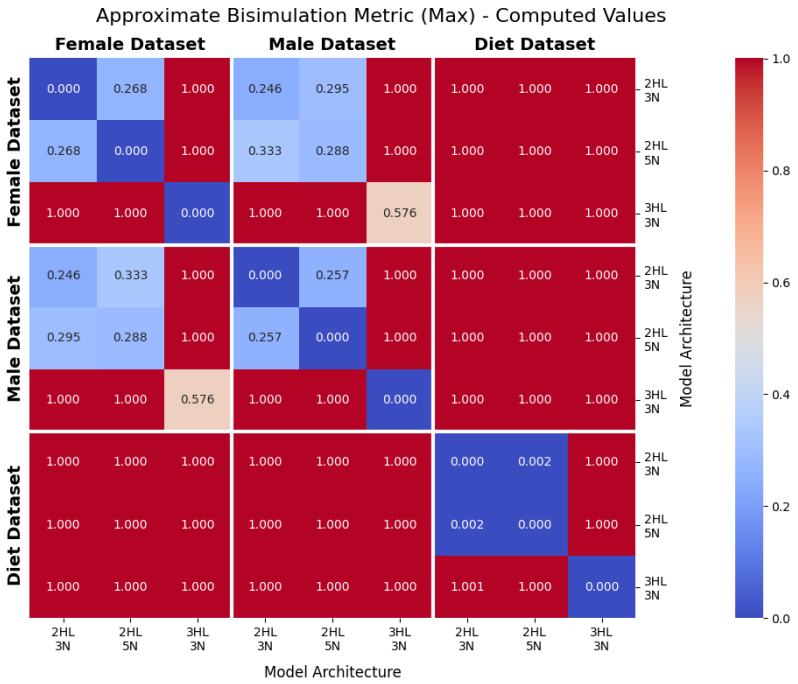

Figure 4.4: Results from the Maximum Approximate Bisimulation distance applied to models trained on variations of the *Stroke* dataset.

in the network topology than the SMT-based metric and, in particular, to the length of paths of the produced PFSP. The strong similarity between models trained on *Diet* dataset having two hidden layers follows the pattern arising from the SMT-based metric, denoting a correlation of the bisimilarity with the output divergence of the models.

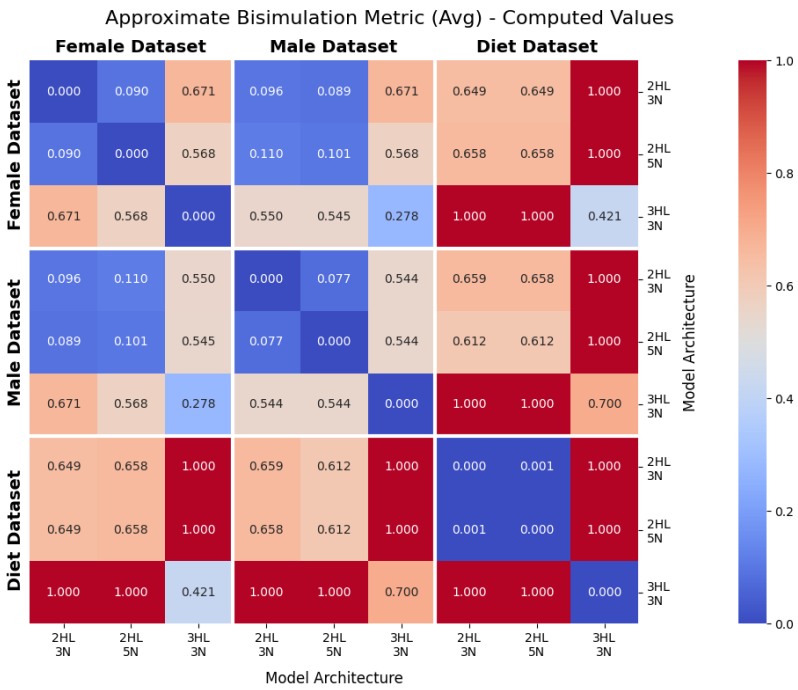

Figure 4.5: Results from the Average Approximate Bisimulation Distance Applied to Models Trained on Variations of the *Stroke* Dataset.

The **Avg** form, on the other hand, provides more evenly distributed values, with the highest values occurring only in case of models having a different depth of the network. It is also worth noting that models computing similar functions with deep architectures tend to be evaluated as more distant than those with shallow architectures, as can be observed from the increasing values of the distance between models trained on *Female* and *Male* dataset (0.096, 0.101, 0.278).

The heatmaps in Figures 4.6 and 4.7 illustrate the registered execution times of the Approximate Bisimulation algorithm in the two forms.

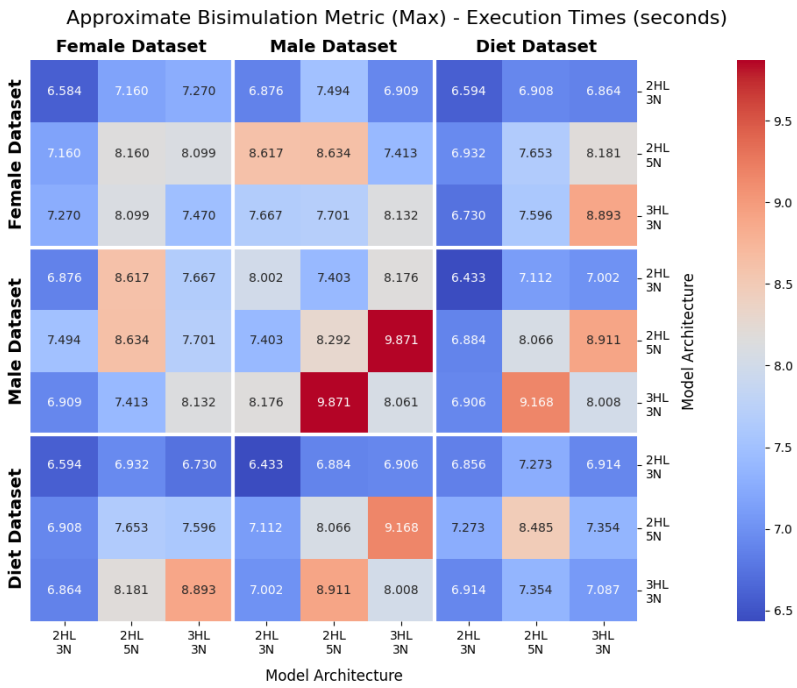

Figure 4.6: Execution Times from the Maximum Approximate Bisimulation Distance Applied to Models Trained on Variations of the *Stroke* Dataset.

It is very important to note that these times comprise the time required to convert the models into PFSPs, which required most of the time registered, while the time required to compute the distance between the two PFSPs always remained in the order of milliseconds (typically 2), even for the most complex models.

In fact, in order to maintain an high precision in the percentage of transitions in the PFSPs, the conversion procedure involved the generation of 100.000 random input samples for each model, which were then used to simulate the models and generate the corresponding PFSPs. For this reason, it can be observed that, despite occasional peaks, the execution time is directly proportional to the number of neurons in the models for both forms of the metric. We explain this behavior by the fact that the number of neurons in the models is directly correlated with the total amount of forward passes per input sample required to generate the PFSPs.

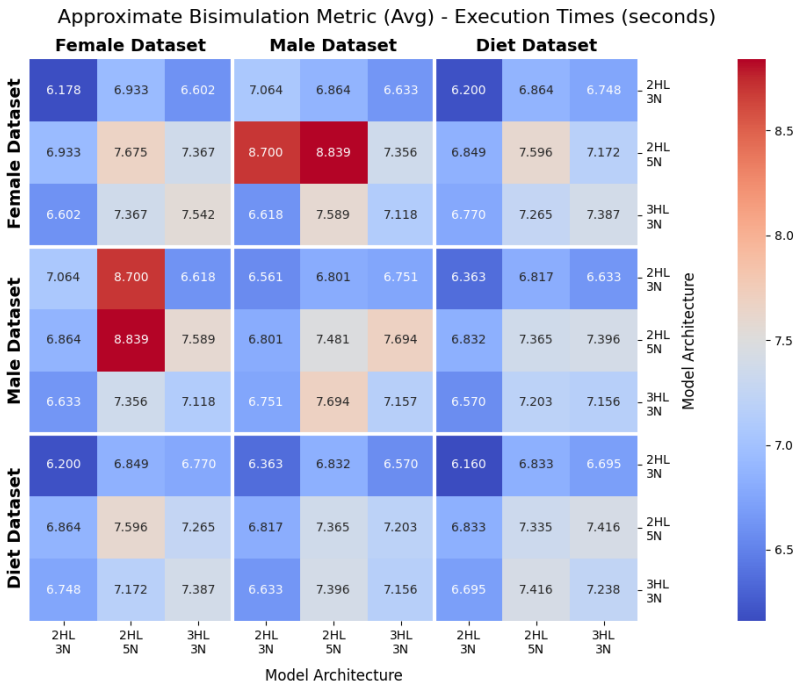

Figure 4.7: Execution times from the Average Approximate Bisimulation distance applied to models trained on variations of the *Stroke* dataset.

Part III
Epilogue

Conclusion

In this work, we explored the problem of semantic neural network comparison evaluating methodologies from existing literature in the field of model checking and formal modeling of system, while adapting them to the specific context of machine learning.

From previous works on the application of SMT Solvers for studying input-output relations of MLPs, we provided a metric definition and an algorithm for computing the distance between two models based on the concept of maximum divergence in the output space. Results from the application of the metric to a pool of models trained on different datasets and architectures showed that the metric is capable of capturing the semantic differences between the models, with the divergence being primarily influenced by the dataset used for training and secondarily the architecture of the models. Despite the promising results, the methodology showed to be computationally demanding, with execution times increasing exponentially with the complexity of the models, making it unsuitable for large-scale applications.

We defined a conversion procedure to interpret feed-forward neural networks as probabilistic FSP and deterministic SPA based on the concept of flow of information through the network. We proposed a possible interpretation of state and we provided two possible methods for computing such representation, the first relying on proving the possibility of transitions though SMT solving, and the second based on the approximation of the transition probabilities through random sampling of the input space.

We used such definition in combination with the concept of approximate bisimulation to define a distance for the comparison of neural network models, along with a novel algorithm for the computation of the distance. Results from the application of the metric to the same pool of models showed that the metric is capable of capturing semantic differences between the models, with the distance being highly sensible to the architecture of the models and the depth of the network.

Finally, we provided an optimized adaptation of the existing algorithm for the computation of the behavioral distance between SPAs by restricting the applicability of the algorithm to the specific case of deterministically defined distributions of target states. We also provided a mathematical adaptation of the algorithm description

The execution times of the algorithm were significantly lower than the SMT-based metric and are linked to a linear-time complexity with respect to the size of the models. This important aspect makes the approach suitable for real-world applications, as we believe it can be applied to large-scale models.

for facilitating its execution whenever SMT solvers like Z3 are used. Despite the proposed improvement, the implementation of the algorithm proved it to be computationally unfeasible for the available technology even for small models, due to the high complexity of the algorithm and the high number of constraints generated.

As the approaches consisting in model checking techniques applied to machine learning models are still in its infancy and are emerging as a promising frontier in computer science, we believe that the results of this work highlight the importance of considering computational efficiency when designing algorithms for the comparison of neural network models, as the complexity of the models and the size of the datasets are rapidly increasing in the field of machine learning. Nevertheless, we believe that the proposed methodologies represent a significant step forward in the development of novel metrics which can provide helpful insights into the explainability and interpretability of neural network models, and we hope that the developed tools and results of this work can inspire further research in the field.

FUTURE WORK

Future work may include further improvement, optimization and enhancing of the presented methodologies, as well as the development of more sophisticated and efficient algorithms, especially those relying on the usage of SMT solvers. We also envision the application of the proposed metrics to different types of models, like RNNs and CNNs, as a useful possible future development of the project.

Abbreviations

SMT Satisfiability Modulo Theory

NN Neural Network

TS Transition Systems

MTS Metric Transition System

FSP Finite State Process

NFSA Non-Deterministic Finite State Automaton

DNN Deep neural Network

ReLU Rectified Linear Unit

CNN Convolutional Neural Networks

KS Kripke Structure

SAT Boolean Satisfiability Problem

NP Nondeterministic Polynomial-Time

MSL Many-Sorted Logic

OSL Order-Sorted Logic

FOLPT First Order Logic with Parametrical Types

BMC Bounded Model Checking

AQTS Action-Labelled Quantitative Systems

SPA Simple Probabilistic Automata

MDP Markov Decision Process

CEGIS Counterexample-Guided Inductive Synthesis

BDT Binary Decision Tree

RL Reinforcement Learning

LMP Labelled Markov Process
LTS Labelled Transition Systems

RNN Recurrent Neural Network

DSPA Deterministic Simple Automata
DFA Deterministic Finite Automata

PFSP Probabilistic Finite State Process

MLP Multi-Layer Perceptron

DFS Depth-First Search

Bibliography

[1] Alessandro Abate, Mirco Giacobbe, and Yannik Schnitzer. *Bisimulation Learning*. 2024. arXiv: 2405.15723 [cs.LO]. URL: https://arxiv.org/abs/2405.15723.

[2] Charu Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Jan. 2018. ISBN: 978-3-319-94462-3. DOI: 10.1007/978-3-319-94463-0.

[3] Aws Albarghouthi. "Introduction to Neural Network Verification". In: *Foundations and Trends® in Programming Languages* 7.1–2 (2021), pp. 1–157. ISSN: 2325-1107. DOI: 10.1561/2500000051.

[4] Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. Cambridge, Mass.: MIT Press, 2008. URL: http://search.ebscohost.com/login.aspx?direct=true&scope=site&db=nlebk&db=nlabk&AN=226091.

[5] Pablo Samuel Castro. "Scalable methods for computing state similarity in deterministic markov decision processes". In: Cited by: 67. 2020, pp. 10069–10076. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85095257064&partnerID=40&md5=b3f8e54c49e52392a84530cca7d387f2.

[6] Taolue Chen, Tingting Han, and Jian Lu. "On metrics for probabilistic systems: Definitions and algorithms". In: *Computers & Mathematics with Applications* 57.6 (2009), pp. 991–999.

[7] Edmund M. Clarke et al. *Handbook of Model Checking*. 1st. Springer Publishing Company, Incorporated, 2018. ISBN: 3319105744.

[8] Yuxin Deng et al. "Metrics for Action-labelled Quantitative Transition Systems". In: *Electronic Notes in Theoretical Computer Science* 153.2 (2006). Proceedings of the Third Workshop on Quantitative Aspects of Programming Languages (QAPL 2005), pp. 79–96. ISSN: 1571-0661. DOI: https://doi.org/10.1016/j.entcs.2005.10.033. URL: https://www.sciencedirect.com/science/article/pii/S157106610600260X.

[9] Josée Desharnais, François Laviolette, and Sami Zhioua. "Testing probabilistic equivalence through Reinforcement Learning". In: *Information and Computation* 227 (2013), pp. 21–57.

[10] Charis Eleftheriadis et al. "On Neural Network Equivalence Checking Using SMT Solvers". In: *Formal Modeling and Analysis of Timed Systems*. Ed. by Sergiy Bogomolov and David Parker. Cham: Springer International Publishing, 2022, pp. 237–257. ISBN: 978-3-031-15839-1.

[11] Antoine Girard and George J. Pappas. "Approximation Metrics for Discrete and Continuous Systems". In: *IEEE Transactions on Automatic Control* 52.5 (2007), pp. 782–798. DOI: 10.1109/TAC.2007.895849.

[12] Dario Guidotti, Laura Pandolfo, and Luca Pulina. "Verifying Neural Networks with Non-Linear SMT Solvers: a Short Status Report". In: *2023 IEEE 35th International Conference on Tools with Artificial Intelligence (ICTAI)*. 2023, pp. 423–428. DOI: 10.1109/ICTAI59109.2023.00068.

[13] Paris C. Kanellakis and Scott A. Smolka. "CCS expressions, finite state processes, and three problems of equivalence". In: *Proceedings of the Second Annual ACM Symposium on Principles of Distributed Computing*. PODC '83. Montreal, Quebec, Canada: Association for Computing Machinery, 1983, pp. 228–240. ISBN: 0897911105. DOI: 10.1145/800221.806724. URL: https://doi.org/10.1145/800221.806724.

[14] Guy Katz et al. "Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks". In: Feb. 2017. ISBN: 978-3-319-63386-2. DOI: 10.1007/978-3-319-63387-9_5.

[15] Sara Mohammadinejad et al. "DiffRNN: Differential Verification of Recurrent Neural Networks". In: *International Conference on Formal Modeling and Analysis of Timed Systems*. 2020. URL: https://api.semanticscholar.org/CorpusID:220646893.

[16] *MS Windows NT Kernel Description*.

[17] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. URL: http://neuralnetworksanddeeplearning.com/.

[18] Pavithra Prabhakar. "Bisimulations for Neural Network Reduction". In: *Verification, Model Checking, and Abstract Interpretation*. Ed. by Bernd Finkbeiner and Thomas Wies. Cham: Springer International Publishing, 2022, pp. 285–300. ISBN: 978-3-030-94583-1.

[19] Francesco Ranzato and Francesco Tapparo. "Computing Stuttering Simulations". In: *CONCUR 2009 - Concurrency Theory*. Ed. by Mario Bravetti and Gianluigi Zavattaro. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 542–556. ISBN: 978-3-642-04081-8.

[20] Siddharth Sharma, Simone Sharma, and Anidhya Athaiya. "Activations Functions in Neural Networks". In: *International Journal of Engineering Applied Sciences and Technology* 04 (May 2020), pp. 310–316. DOI: 10.33564/IJEAST.2020.v04i12.054.

[21] Cheng Wang, Carolin Lawrence, and Mathias Niepert. "State-Regularized Recurrent Neural Networks to Extract Automata and Explain Predictions". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.6 (2023), pp. 7739–7750. DOI: 10.1109/TPAMI.2022.3225334.

[22] Gail Weiss, Yoav Goldberg, and Eran Yahav. "Extracting automata from recurrent neural networks using queries and counterexamples". In: *Machine Learning* 113.5 (2024), pp. 2877–2919. ISSN: 1573-0565. DOI: 10.1007/s10994-022-06163-2. URL: https://doi.org/10.1007/s10994-022-06163-2.

[23] Simon Wiedemann et al. "DeepCABAC: A Universal Compression Algorithm for Deep Neural Networks". In: *IEEE Journal of Selected Topics in Signal Processing* 14.4 (2020), pp. 700–714. DOI: 10.1109/JSTSP.2020.2969554.

[24] Weiming Xiang and Zhongzhu Shao. "Approximate Bisimulation Relations for Neural Networks and Application to Assured Neural Network Compression". In: *2022 American Control Conference (ACC)*. 2022, pp. 3248–3253. DOI: 10.23919/ACC53348.2022.9867845.